

PRINT-FLOW-360 · RESEARCH LIBRARY

Retention Lifecycle

6 research documents · each with plain-language key takeaways

Compiled 2026-06-26

Table of Contents

1. Retention & Lifecycle — Master Index & Cross-Cutting Roadmap

2. Community Building as a Retention Moat for Print-Flow-360

3. Customer Success & Onboarding for Non-Technical Store Owners

4. Dunning & Involuntary Churn Recovery — Failed/Expired-Card Payment Recovery on SaaS Subscriptions

5. Lifecycle & Behavioral Email (Triggered Journeys) for Print-Flow-360

6. NPS / CSAT / Customer Health Scoring & At-Risk Alerting

Retention & Lifecycle — Master Index & Cross-Cutting Roadmap

Source: [readme/research/retention-lifecycle/RETENTION_AND_LIFECYCLE_MASTER_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a plan for keeping customers — both store owners who pay for Print-Flow-360, and the shoppers who buy from those stores. Losing a customer costs far more than keeping one, so this plan maps out what to build, in what order, to stop customers from leaving. Nothing in this plan has been built yet — it is a research and planning document.

The main parts explained simply:

- **Onboarding (getting new store owners started)** — Right now, a store owner is marked “done with setup” even if they have never received a real order. This section fixes that: a store owner is only truly “started” when their store is live and they have received their first order. It also proposes a “place a test order” shortcut so new owners can see the full experience within 24 hours.
- **Lifecycle emails (sending the right message at the right time)** — Today the platform can send bulk email campaigns, but it cannot automatically send a follow-up email hours after someone abandons their shopping cart, or nudge a customer who has not bought in months. This section plans a “journey engine” — a system that watches what customers do and sends the right message at the right moment, without anyone pressing a button.
- **Dunning (recovering failed payments)** — When a store owner’s subscription payment fails, nothing happens today — the failure is invisible to the platform. This section plans a proper process: warn owners before their card expires, retry the charge over several days, send plain-language emails asking them to update their card, and only suspend their account as a last resort. Good dunning recovers more than half of all failed payments.
- **Health scoring and feedback surveys (spotting trouble early)** — This section plans a simple “health score” for both store owners and their shoppers: a traffic-light system (green/yellow/red) based on activity. A red score triggers an alert so someone can step in before the customer leaves. It also plans short, one-question satisfaction surveys sent automatically after an order is delivered.
- **Community features (giving store owners a reason to stay)** — This section plans two things: a public place where store owners can vote on features they want, and a shared library of design templates that any store owner can use. Both take a long time to pay off but make the platform harder to leave.
- **Two shared building blocks** — Almost everything above depends on the same two pieces of foundation: (1) a “journey engine” that can send delayed, behaviour-triggered messages, and (2) a “health signals layer” that scores customers nightly. Building these once means every other feature just plugs in, instead of each feature reinventing its own version.

What to do with this:

Start by switching the background job system from “sync” to Redis (a technical prerequisite that unlocks everything) and fix the payment-failure pipeline so failed charges are visible. These two steps protect revenue immediately and unblock every other item in the plan.

Date: 2026-06-16 · **Status:** Research-only. Nothing in this master or its five linked docs has been built. This is a planning spine, not a changelog. **Theme:** How Print-Flow-360 keeps the customers it

already has — both the **tenants** it sells the SaaS to, and the **end-customers** those tenants sell print to. Retention is cheaper than acquisition and compounds into NRR; this master ties five independent research streams into one sequenced plan so we build shared infrastructure once instead of five times.

TL;DR

Five 2026-06-16 research docs each tackle a slice of retention and lifecycle: **onboarding/activation, lifecycle & behavioral email, dunning / involuntary churn, NPS/CSAT & health scoring**, and a **community moat**. Read individually they look like five separate builds. Read together, they collapse onto **two shared platforms** — a triggered **journey/automation engine + event bus**, and a **health-score/signals layer** — that nearly every topic depends on. The revenue-protective work (onboarding activation, dunning) is P0; the shared lifecycle-email engine is the P1 platform that unblocks NPS delivery, win-back, and nudges; health scoring is P1; community is the P2 long-horizon NRR play. Everything is framed by **two audiences**: the platform retaining tenants (Audience-1) and store owners retaining their end-customers (Audience-2) — and the docs are strict about never letting Audience-1 plumbing leak into Audience-2 tenant-scoped engines (or vice versa).

Index — the five docs

#	Doc	One-line hook
1	<code>RETENTION_CUSTOM ER_SUCCESS_ONBOAR DING_2026-06- 16.md</code>	The setup checklist exists but onboarding “done” stops at setup, not the first-order activation event — and its state lives in browser localStorage with zero landlord visibility.
2	<code>RETENTION_LIFECY CLE_BEHAVIORAL_EM AIL_2026-06- 16.md</code>	Strong broadcast campaigns + a one-shot automation engine, but no journey/drip engine, no behavioral triggers (abandoned cart, win-back), and no per-recipient enrolment or frequency cap.
3	<code>RETENTION_DUNNIN G_INVOLUNTARY_CHU RN_2026-06-16.md</code>	Stripe retries cards, but nothing local consumes the failure webhooks — <code>charge_failed</code> is never written, <code>BillingService::renew()</code> is never scheduled, and there’s no card-expiry warning or self-serve card update.
4	<code>RETENTION_NPS_CS AT_HEALTH_SCORING _2026-06-16.md</code>	No survey or health-score spine for either audience today; the Action Center rule engine + a 90-day at-risk proxy + existing delivery rails are a strong foundation for a shared survey spine and two HealthScoreService classes.
5	<code>RETENTION_COMMUN ITY_MOAT_2026-06- 16.md</code>	The two hardest raw materials for a community moat already exist (the designer template engine over one shared <code>designer_documents</code> table, and a half-built <code>PlatformInquiry</code> intake) — turning them into a community template library + public roadmap/voting is mostly additive.

The two-audiences lens (the organizing frame)

Every retention decision in this master sorts into exactly one of two audiences. Confusing them is the recurring failure mode the docs warn against.

	Audience-1 — Platform ↔ Tenant	Audience-2 — Store owner ↔ End-customer
Who churns	The store owner (our paying SaaS tenant)	The shopper buying print from a tenant's store
Churn metric	Logo churn / NRR / MRR	Repeat-purchase rate / customer LTV
Owned by	Landlord / System jobs (<code>app/Jobs/System</code> , <code>resources/views/emails/system</code>)	Tenant-scoped engines (<code>Automation</code> , <code>Journey</code> , campaigns)
Topics that serve it	Onboarding activation, dunning, tenant health, community	Lifecycle/behavioral email (abandoned cart, win-back), customer health, NPS/CSAT to shoppers
Hard rule	Never run landlord dunning/nudges through the tenant <code>Automation</code> engine or <code>PAYMENT_FAILED</code> template (wrong audience)	Never let cross-tenant community data leak past <code>BelongsToTenant</code> ; never show tenant UUIDs as authorship

Several topics have **both** an Audience-1 and an Audience-2 face (notably health scoring and lifecycle journeys). The plan keeps them as separate code paths sharing a common substrate, never one engine straddling both.

Cross-cutting view — build once, reuse many

The five docs independently converge on the same two pieces of shared infrastructure. Building these first turns four of the five topics from “new system” into “new caller.”

Shared platform A — Triggered journey/automation engine + event bus

The single highest-leverage shared build. It extends the existing one-shot automation substrate (reuse `ActionExecutor.php` and its `[token]` parser **verbatim**) into a multi-step, delayed, per-recipient journey engine.

Depends on it:

- **Doc 2 (lifecycle email)** — *is* this engine: `Journey` / `JourneyStep` / `JourneyEnrolment` (HasUuid + BelongsToTenant), `ProcessJourneyStepJob` with delayed dispatch, abandoned-cart recovery as the flagship journey.
- **Doc 4 (NPS/CSAT)** — delivers transactional surveys via the same rails: add an `order_delivered` trigger, embed the first question in email; no new delivery engine.

- **Doc 1 (onboarding)** — proactive nudge sequences (Day 3 / Day 7 stalled-onboarding) ride the same trigger/job pattern, on the **landlord** side as System jobs.
- **Doc 3 (dunning)** — the landlord dunning email sequence is structurally the same delayed-step pattern, again as System jobs (Audience-1), explicitly **not** the tenant journey engine.
- **Doc 5 (community)** — close-the-loop voter notifications and changelog posts ride the same notification rails.

Two hard prerequisites called out by multiple docs:

1. **QUEUE=sync** → **Redis**. Delayed steps are *impossible* on the sync driver. Docs 2, 3, and 5 all gate their first shippable journey/job on this. Do it first.
2. A shared **CommunicationGuard** (**frequency-cap + suppression**). Consulted by campaigns, automations, AND journeys before any non-transactional send; enforces a per-recipient monthly cap and 60–90 day sunset (**EmailSuppression** + **EmailSendLedger**). Transactional sends (dunning, order, NPS-on-delivery) are exempt. Build it alongside the engine so the first journey can't spam.

Shared platform B — Health-score / signals layer

A weighted-signal scoring service that persists nightly bands and drives Action Center alerts — built **twice** (one per audience) but as one pattern.

Depends on it / feeds it:

- **Doc 4** owns the build: **CustomerHealthService** (Audience-2: Activity 40 / Engagement 30 / Milestones 20 / Recency 10, bands Green 75–100 / Yellow 40–74 / Red 0–39) and **TenantHealthService** (Audience-1).
- **Doc 1** asks for exactly a **TenantHealthService** for first-90-day at-risk detection — same service Doc 4 specifies. Build once.
- **Doc 3** contributes a signal both ways: a **charge_failed** / card-failure event is 20–40% of churn and must fold into both health scores.
- **Doc 4** evolves the existing **AtRiskCustomersRule.php** to read the persisted band instead of the lone 90-day recency check — richer at-risk feed, zero new frontend.

Smaller shared primitives

- **Event taxonomy** — **cart_abandoned** , **order_delivered** , **first_order_placed** , **customer_dormant** , **order_delivered** (NPS), **charge_failed** , **onboarding_step_completed** , **first_order_at** . Defining these once in **TriggerTypeEnum** (and the System-side equivalents) lets journeys, surveys, health signals, and Action Center rules all subscribe.
 - **Action Center as the universal “now what”** — onboarding (**OnboardingIncompleteRule**), at-risk customers (health band), and detractor NPS responses all surface through the existing rule engine. One alert surface, many producers.
-

Unified phased roadmap (P0 → P2, sequenced across all five docs)

Ordering principle: **protect revenue first**, then **build the shared platform that unblocks everything**, then **instrument**, then **moat**. Within a phase, dependencies dictate order.

P0 — Revenue-protective + foundations (do these first)

Item	Doc	Audience	Why now
Move <code>QUEUE=sync</code> → Redis	2,3,5	infra	Hard prerequisite for every delayed job/journey below. Nothing lifecycle ships without it.
Persist onboarding state server-side (<code>StoreOnboardingProgress</code> + <code>OnboardingProgressService</code> , per-step timestamps, <code>first_order_at</code>); rewire <code>useSetupChecklist.ts</code> off localStorage	1	A1	Fixes per-device reset; unlocks landlord CS visibility + every onboarding metric.
Redefine “done” = store-live AND first-order; add non-dismissible “Receive your first order” milestone; kill the 30-day hard expiry while not-live	1	A1	The actual activation event. Stops counting setup as success.
“Place a test order” path (reuse seeded demo product + <code>WELCOME10</code> + offline payment; extend <code>is_demo</code> to orders)	1	A1	Time-to-first-value <24h without waiting on a real shopper.
Make subscription failures observable — extend <code>PaymentEventDispatcher</code> to consume Stripe <code>invoice.payment_failed/paid</code> ; finally write <code>Subscription.status='charge_failed'</code> ; add the missing <code>subscriptions.gateway_subscription_id</code> migration	3	A1	Today the failure path is dead. Can't dun what you can't see.
<code>SubscriptionStateService</code> (active → past_due → grace → suspended → cancelled) with configurable 7–14 day grace, soft-vs-hard suspension, cache invalidation, audit logs/events	3	A1	Replaces the 1-day inline grace; the spine all dunning hangs off.

P1 — The shared lifecycle platform + instrumentation

Item	Doc	Audience	Why here
Journey engine — <code>Journey</code> / <code>JourneyStep</code> / <code>JourneyEnrolment</code> extending the automation substrate, reusing <code>ActionExecutor</code> + token parser; <code>ProcessJourneyStepJob</code> delayed dispatch; <code>journeys:tick</code> cron	2	A2	The shared platform (A). Unblocks abandoned-cart, win-back, NPS delivery.
Lifecycle triggers in <code>TriggerTypeEnum</code> (<code>cart_abandoned</code> , <code>order_delivered</code> , <code>first_order_placed</code> , <code>customer_dormant</code>) with de-dup enrolment + exit conditions	2	A2	The event taxonomy every downstream subscriber needs.
Abandoned-cart recovery as the flagship journey (3 steps ~1h/24h/72h, exit on <code>order_placed</code>), seeded DISABLED	2	A2	~76% of automation revenue; highest single-gap ROI.
CommunicationGuard frequency-cap/suppression layer	2	both	Gate before the first journey can spam; reused by campaigns + automations.
Pre-dunning: <code>NotifyCardExpiringJob</code> (mirror <code>NotifyTrialExpiringJob</code> , ~30 days before <code>expiration_date</code> , one-click update-card)	3	A1	Highest-ROI/lowest-effort dunning; data already stored.
Print-Flow-owned retry loop — <code>RetrySubscriptionChargeJob</code> + <code>billing:run-dunning</code> (Day 1/3/5/7), gated by per-gateway capability flag (trust Stripe Smart Retries; drive Razorpay/PayPal/AuthorizeNet/PayTM) + 4-step dunning emails + self-serve recovery page	3	A1	Reclaims the 50–65% of involuntary churn good dunning recovers.
Landlord-side journeys as System jobs — new-tenant welcome/activation, go-live nudges, stalled-onboarding Day 3/7 (<code>NotifyStalledOnboardingJob</code> + <code>ONBOARDING_NUDGE</code> template), dunning sequence	1,2,3	A1	The tenant engine can't serve Audience-1; these are separate scheduled jobs.
Shared survey spine — <code>Survey</code> + <code>SurveyResponse</code> (UUID, <code>BelongsToTenant</code> , Postgres CHECK enums), <code>SurveyService</code> computing NPS/CSAT/CES + persistence Feature test (silent-lie rule)	4	both	The instrumentation layer; delivered via the P1 journey rails.
Survey delivery via existing rails — <code>order_delivered</code> trigger for transactional NPS (first question embedded in email); <code>SendTenantOnboardingCsatJob</code> 7–10 days after <code>seedAll</code>	4	both	No new engine; reuses platform A.
CustomerHealthService (signal-stack, nightly persist to <code>customer_health_scores</code> , bands) + evolve <code>AtRiskCustomersRule</code> to read the band	4	A2	Shared platform B, Audience-2 side; richer at-risk feed, no new UI.
TenantHealthService for SaaS churn; fold <code>charge_failed</code> into both scores; wire detractor ($NPS \leq 6$) responses into Action Center + Activity logs	1,3,4	A1	Platform B, Audience-1 side — satisfies Doc 1's request too. Score

Item	Doc	Audience	Why here
			change always triggers an action.
In-app Help slide-over + step explainers + 3–5-stop first-run tour ; replace 30-day expiry with Action Center escalation (<code>OnboardingIncompleteRule</code>)	1	A1	Self-serve activation support; closes the onboarding loop.

P2 — Long-horizon moat (12–18 month payoff)

Item	Doc	Audience	Why last
Public Roadmap + Feature Voting — extend <code>platform_inquiries</code> (plain-language status enum) + new <code>feature_votes</code> table + <code>votes_count</code> ; <code>RoadmapService</code> ; one-click vote; mandatory close-the-loop voter notifications	5	A1	Highest-ROI community primitive; data model half-built; rides P1 notification rails.
Cross-Tenant Community Template Library on <code>DesignerTemplate</code> — additive <code>is_published_to_community</code> / <code>author_tenant_id</code> / <code>community_status</code> (default-false = byte-identical), <code>CommunityTemplateService</code> bypassing <code>BelongsToTenant</code> via <code>withoutGlobalScope</code> , clone-to-account, <code>usage_count</code> ranking	5	A1	Cheap (one shared <code>designer_documents</code> table) but value compounds slowly.
Moderation + silent-leak test shipped WITH the library — pending/approved/rejected, landlord review queue, store-name attribution (never tenant UUIDs), explicit cross-tenant invisibility test	5	A1	Non-negotiable safety; not a follow-up.
Changelog + peer Q&A/comments on community templates (~73% support-deflection potential), Ambassador badge from <code>usage_count</code> ; then anonymized cross-tenant benchmarking metrics via nightly <code>routes/console.php</code> command	5	A1	Longest horizon; feeds the (absent) support spine.
DEFER standalone forum (Discord/Slack/Circle) — validate any chat community off-platform first	5	A1	Community needs 12–18 months; lead with in-product primitives.

Success metrics rollup

North Star (Audience-1): % of new tenants reaching store-live + first order within 7 days — the activation event Doc 1 redefines “done” around. Retention compounds from this single moment.

Supporting North Star (Audience-2): end-customer repeat-purchase rate within tenant stores — the downstream metric the lifecycle engine, win-back journeys, and customer health scores all move.

Topic	Primary metric(s)
Onboarding / CS (Doc 1)	Activation rate (live + first order ≤7d); time-to-first-value (<24h target via test order); % tenants Activated vs Stalled vs At-Risk; first-90-day churn.
Lifecycle / behavioral email (Doc 2)	Abandoned-cart recovery rate (target ~the ~76%-of-automation-revenue gap); journey-attributed revenue; per-recipient send frequency within cap; win-back reactivation rate.
Dunning / involuntary churn (Doc 3)	Involuntary-churn recovery rate (target 50–65%); failed-payment → recovered %; card-expiry pre-dunning save rate; dunning email → update-card conversion.
NPS/CSAT/Health (Doc 4)	NPS / CSAT / CES scores + response rate (email-embedded first question lift); health-band distribution (Green/Yellow/Red) and band-transition rate; detractor → Action Center → resolved loop.
Community moat (Doc 5)	Feature-vote participation + close-the-loop notification rate; community templates published/cloned; <code>usage_count</code> -ranked adoption; support-deflection % from peer Q&A; tenant NRR lift over 12–18 months.

How to use this master

1. **Start at P0.** `QUEUE→Redis` first, then the onboarding-activation and dunning-observability builds — they protect revenue and unblock everything.
2. **Build the two shared platforms (A: journey engine + event bus, B: health/signals) once.** Resist building topic-specific versions; four of five topics are callers, not owners.
3. **Keep the two audiences in separate code paths** sharing the substrate — never one engine straddling both, never landlord plumbing in the tenant engine.
4. **Read the source doc before building any item** — each links to concrete models, services, routes, and file paths following the codebase's UUID / `BelongsToTenant` / service-layer / silent-lie-test conventions. This master sequences; the source docs specify.

Community Building as a Retention Moat for Print-Flow-360

Source: [readme/research/retention-lifecycle/RETENTION_COMMUNITY_MOAT_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document explains how building a small “community” around Print-Flow-360 can stop print-shop owners from cancelling their subscriptions. The core idea is simple: when store owners feel heard, can share useful designs with each other, and can see their feedback turn into real features, they stay longer — and that directly means more revenue.

The main parts explained simply:

- **Why community keeps customers** — Research shows that when customers can vote on features and see those features get built, they are far less likely to leave. A simple voting board can improve how long people stay by a meaningful amount. This matters now because keeping existing customers is cheaper than finding new ones.
- **Two types of users — and why they must not be mixed up** — This platform has two separate relationships: Print-Flow-360 talking to print-shop owners (the people who pay for the platform), and print-shop owners talking to their own customers (the shoppers who buy prints). All the ideas in this document are about the first relationship only. Do not mix them up.
- **Public Roadmap and Feature Voting** — A page where store owners suggest features and vote on each other's ideas. When a feature gets built, the platform emails everyone who voted. That notification is the key step — it tells store owners “we heard you, and we delivered.” This is the fastest win with the least effort to build.
- **Shared Template Library** — A gallery where store owners can share their print design templates with other stores on the platform. When one owner publishes a template and another copies it, both get value. Print shops already spend their days reusing and adapting designs, so this feels natural and useful. This is described as the single biggest reason customers would stay.
- **What already exists in the code** — The platform already has a half-built feature-request system and a full design template engine. Building the roadmap and template library means extending what is already there, not starting from scratch. That makes both tasks faster than they might seem.
- **What is missing and needs to be built** — There is currently no way for store owners to vote on features, no public roadmap, no cross-store template gallery, and no notification when their idea gets built. Also, there is no way for store owners to help each other with questions (no peer support). These are the gaps this plan wants to close.
- **Phase-2 ideas (build later)** — After the roadmap and templates are live, the plan adds an in-product “what's new” feed, the ability to comment on templates and get answers from other owners, and eventually an anonymised comparison showing how a store performs versus others on the platform.
- **One technical requirement before anything ships** — All the email notifications that make this work need a proper background job queue (called Redis). Without it, notifications would be slow and unreliable. This must be set up first.

What to do with this: Start with the voting roadmap first — it takes the least effort and delivers the most immediate trust-building. Then build the shared template library, which is the strongest reason for a print-shop owner to stay on the platform long-term. Do both before adding anything more complex.

Date: 2026-06-16 **Status:** Research-only — nothing built yet. This document is a plan, not a record of shipped work. No migrations, services, or UI described below exist; every “Recommended” item is a proposal for an engineer to implement later.

TL;DR

Print-Flow-360 already owns the two hardest-to-build raw materials for a community-driven retention moat — a full **designer template engine** (`app/Models/DesignerTemplate.php` over a *single shared* `designer_documents` Postgres table) and a half-built **feature-request intake** (`app/Models/PlatformInquiry.php` with `type='feature_request'`). Both are scoped/locked today, but converting them into a **cross-tenant Community Template Library** and a **public Roadmap + Feature Voting board** is mostly additive work (opt-in flags + a votes table + a scope bypass), not new infrastructure. Community is the *longest-horizon* moat in SaaS (12–18 months to payoff per CLG research), so we phase it: ship the two low-effort, high-retention primitives first (roadmap voting, template sharing), close the loop with notifications and a changelog, then layer peer Q&A and — last — a benchmarking data-network-effect. **Do not build a standalone forum first.** Keep the two audiences strictly separated (platform ↔ tenant retention vs. store-owner ↔ end-customer storefront tooling).

1. Why this matters for a non-technical-store-owner print SaaS

Print-Flow-360's primary users are **non-technical print-shop owners** who navigate by intuition and rarely read docs (per `CLAUDE.md §0`). Three structural facts make community an unusually strong retention lever *for this specific product*:

1. **There is no support spine yet.** Per prior QA (`readme/REAL_USER_QA_READINESS_2026-06-15.md`), there is no ticket system. Every question today has nowhere to go. A peer/community surface deflects an estimated ~73% of inquiries (Higher Logic, 2026) — for a product with *zero* formal support capacity, peer deflection isn't a nice-to-have, it's the cheapest path to scalable support.
2. **Print shops are templating businesses.** Their core daily act is reusing and adapting designs. A shared template gallery maps directly onto how they already work — it is not an abstract “community,” it is “more templates, for free, that I can make my own.” That's an intuitive value proposition a non-technical owner grasps without explanation.
3. **Retention/NRR is where the money is right now.** B2B SaaS averages ~3.5% monthly churn (top performers <2%), ~74% annual retention, and top-quartile NRR >120% (Orb Billing; Vitally, 2025–2026). With acquisition spend down ~3.3% market-wide, expansion/retention moats command premium valuations. **Community here is a retention/NRR play, not an acquisition gimmick** — frame every decision against “does this make a store owner less likely to leave?”

The unifying principle from `CLAUDE.md §0` : every surface must be **best-code AND best-UX** for a shopkeeper. A roadmap status must read “Planned” / “Shipped”, never `in_progress` . A template author must read “by Acme Print Co.”, never a tenant UUID.

2. The two audiences — and which one this work serves

This is the single most important conceptual guardrail. The codebase contains assets for **both** relationships, and conflating them will waste effort.

	Audience 1 — Platform ↔ Tenant (store owner)	Audience 2 — Store owner ↔ their End-customers
Who	Print-Flow-360 (landlord) talking to the print-shop owners who pay for it	A print shop talking to the shoppers buying on its storefront
Retention goal	Stop <i>store owners</i> from churning off the SaaS	Help a store owner sell more / retain <i>their</i> buyers
Existing assets	<code>PlatformInquiry</code> (feature requests), admin shell <code>nuxt/app/components/admin/layout/Sidebar.vue</code> , landlord routes <code>routes/admin-api.php</code>	<code>UgcGalleryBlock.vue</code> , <code>PortfolioGalleryBlock.vue</code> , storefront CMS blocks
This document targets	✅ YES — all community-moat recommendations are Audience 1	❌ NO — these are storefront marketing tools, reused only as <i>UI patterns</i>

The trap: `UgcGalleryBlock.vue` (“#MadeWithUs” photo grid) and `PortfolioGalleryBlock.vue` (“Our Work”) look like community features. They are **storefront** blocks that a store owner shows *their shoppers*. They are excellent reusable UI patterns for rendering a curated gallery — but they solve Audience 2’s retention, not the platform’s. **Every recommendation below is Audience 1.** Do not wire a tenant-community feature into the storefront block renderer.

3. What already exists in the codebase

3.1 The strongest seed asset — the Designer Template engine

- `app/Models/DesignerTemplate.php` — full template system over the shared `designer_documents` table: preview thumbnails, categories, product binding, `is_default` / `sort_order`, and **account-level** (`store_id = null`) **vs store-level scoping**. This is the natural raw material for a cross-tenant template marketplace.
- `app/Http/Controllers/Api/Designer/AccountDesignerTemplateController.php` — account/tenant-level CRUD. Hard-scoped: explicit `where('tenant_id', ...)` (~line 15) on top of the `BelongsToTenant` global scope.
- `app/Http/Controllers/Api/Store/DesignerTemplateController.php` — store-level management.
- `app/Http/Resources/Api/Storefront/DesignerTemplateResource.php` — storefront-facing resource (a starting point for a community resource).

3.2 Single-database multi-tenancy — makes cross-tenant sharing technically cheap

- `database/migrations/2026_03_16_120000_create_designer_documents_table.php` — `designer_documents` is **one shared Postgres table** with a `tenant_id` column (stancl/tenancy column-scoping, **not** separate tenant databases). Surfacing templates across tenants needs only a **new query scope + opt-in flag** — no cross-database plumbing. The only thing to deliberately bypass is the `BelongsToTenant` global scope (via `withoutGlobalScope`), gated behind an opt-in published flag.

3.3 Public roadmap / feature-voting seed — already half-built (Audience 1)

- `app/Models/PlatformInquiry.php` + `database/migrations/2025_07_13_093009_create_platform_inquiries_table.php` — already has `type='feature_request'` with `feature_category`, `feature_title`, `feature_description`, `urgency_level`, `budget_estimate`, `attachment_path`.
- `nuxt/app/pages/admin/inquiries/index.vue` + `app/Http/Controllers/Api/Admin/Platform/PlatformInquiryController.php` — admin intake surface.
- **Today it is write-only intake:** no public board, no voting, no status lifecycle, no close-the-loop. But the data model for a roadmap already exists.

3.4 Cross-store “system template” precedent — the pattern to copy

- `app/Models/Store/StoreBlockTemplate.php` — already implements the exact moat-relevant shape: an `is_system` flag (platform-provided vs tenant-custom), `usage_count` with `incrementUsage()`, and `system()` / `custom()` scopes, seeded via `app/Services/Onboarding/StoreOnboardingService.php`. This is the blueprint for a curated, popularity-ranked shared library (`is_system` + `usage_count` + `is_published`).

3.5 UGC / showcase CMS blocks — reusable UI pattern (Audience 2, do not conflate)

- `frontstore/app/components/storefront/blocks/UgcGalleryBlock.vue` (#MadeWithUs grid), `PortfolioGalleryBlock.vue` (“Our Work”, lightbox + filters), `ImageBlock.vue`. Registered via `nuxt/app/config/blockTypes.ts` + `nuxt/app/config/blockTypes/`. Prove the platform can render curated galleries — reuse as a *pattern* for a tenant-owner template showcase wall, not as the feature itself.

3.6 Engines to drive participation

- `app/Services/EmailCampaign/` (`EmailCampaignService`, `AudienceFilterResolver`, `CustomerSegmentService.php`) + the per-minute scheduler `email-campaigns:dispatch-scheduled` — reusable to invite owners to events/webinars or notify on roadmap status changes, *once a landlord-side audience equivalent is wired*.
- `app/Services/EmailService.php` + `app/Enums/EmailTemplateEnum.php` — the transactional path for close-the-loop notifications (new template slugs).
- `nuxt/app/components/admin/layout/Sidebar.vue` + `nuxt/app/layouts/admin-sidebar.vue` — where an “Ideas & Roadmap” / “Community Templates” nav entry lives.

4. Gaps (what's missing)

1. **No cross-tenant template/design sharing.** `DesignerTemplate` is hard-scoped by `BelongsToTenant` + explicit `where('tenant_id', ...)` in both controllers. No `is_published` / `is_shared` flag, no community gallery query, no author/attribution field, no clone-into-my-account flow, no moderation. **The single biggest moat lever is unbuilt.**
 2. **No public roadmap or feature voting.** `PlatformInquiry` stores `feature_request` rows but has no votes table, no public board UI, no status lifecycle, no voter notification, no close-the-loop. Customers cannot see their request acknowledged — the exact transparency research links to retention is absent.
 3. **No community/forum/peer-support surface of any kind.** Grep for `forum|community|discussion|ambassador` across `app/Models` and `routes/` returns **zero**. No peer Q&A, no template comments, no store-owner directory, no expert/ambassador role. With no ticket spine either, there is **zero peer deflection** today.
 4. **No benchmarking / peer-comparison feature.** No anonymized cross-tenant aggregate metrics (e.g. “your avg turnaround vs network median”). A classic community-data moat, entirely absent.
 5. **No events/webinar/content infrastructure** for the landlord → tenant relationship. The only proactive platform → tenant touch is `NotifyTrialExpiringJob`; no announcement feed, changelog, or webinar invite mechanism on the landlord side.
 6. **Audience-confusion risk in existing assets.** `UgcGalleryBlock` / `PortfolioGalleryBlock` serve the store owner's **end-customers** (Audience 2), not the platform ↔ owner community (Audience 1). Reusable as UI, but solve a different relationship.
 7. **Queue reliability.** `QUEUE=sync` per project memory. Any notification-driven loop (vote notifications, digests) needs **Redis queue in prod first**.
 8. **No moderation / abuse / content-ownership model** for any cross-tenant UGC (templates, showcase, posts) — a hard prerequisite before any tenant-to-tenant surface ships.
-

5. Best practices & benchmarks (with sources)

- **Community is the longest-horizon moat.** HubSpot, Notion, and Figma communities measurably contribute to *both* acquisition and retention, but payoff requires a **12–18 month** investment in genuine value. Phase accordingly; do not expect quick wins. (*Mental Momentum / SaaS Operations CLG research, 2025.*)
- **Public roadmap + feature voting is the highest-ROI, lowest-effort retention primitive.** “Customers are less likely to churn if they can see their requested feature is on the roadmap.” Acknowledged requests create ownership and emotional connection; a 5% retention lift can mean up to **95% more revenue**. (*Canny; ProductLift; EasyDesk, 2026.*)
- **Close the loop is non-negotiable.** Update post statuses as work moves; tag/notify voters when a feature ships or is declined. **The notification — not the vote — is what retains.** (*Canny, 2026.*)

- **Reduce participation friction.** Anonymous / one-click voting gets **50–70% more participation** than account-gated voting. Embed the board in-product; link from onboarding emails and support; prompt a vote whenever someone requests something. (*Canny vs. FeatureVote.*)
- **Peer community deflects support cost.** ~73% of inquiries can be solved by peer-to-peer content; customers trust peers using the same product; each deflected ticket saves ~\$15–20. Directly relevant given Print-Flow has no support spine. (*Higher Logic; CX Today; IrisAgent, 2026.*)
- **Template/asset marketplaces are a proven UGC network-effect moat.** Airtable and Notion turned users into the primary drivers of growth/value via shared template galleries; users co-create value competitors cannot replicate. (*Mental Momentum CLG research, 2025.*)
- **Ambassadors/experts extend support and act as advocates.** Identify power users (high `usage_count`, high template adoption) and formalize an expert tier. (*Higher Logic; CX Today.*)
- **Retention-first market context.** B2B SaaS avg ~3.5% monthly churn, top <2%, ~74% annual retention, top NRR >120%. Community is a retention/NRR play. (*Orb Billing; Vitally; ever-help, 2025–2026.*)
- **Stage it as a disciplined hybrid.** CLG works only inside a structured GTM, not as a bolt-on “forum.” Start with the narrowest high-value primitive; expand once engagement is proven. (*TheSmarketers CLG B2B 2026; Omnifunnel.*)

6. Recommended architecture/implementation for THIS codebase

All proposals follow `CLAUDE.md` invariants: **UUID PKs** (`HasUuid`), **BelongsToTenant** on tenant-owned models, controller flow `FormRequest → Service → Resource → successResponse()`, business logic in `app/Services/{Module}/`, **no `$fetch` in `.vue`** (composables only), Pinia for state, plain-language UX (no raw IDs/keys), and **additive-only** migrations (default values preserve byte-for-byte current behavior).

6.1 Public Roadmap + Feature Voting (Audience 1) — extend, don’t reinvent

Data (Postgres, additive):

- Add to `platform_inquiries` (migration): `status` — a Postgres `enum`-style column (compiles to a `CHECK` constraint; per `CLAUDE.md §6` widening later = drop constraint then `->change()`). Values: `submitted | under_review | planned | in_progress | shipped | declined`. Default `submitted` so existing rows are unchanged.
- New table `feature_votes` (migration): `uuid` PK (`HasUuid`), `platform_inquiry_id` (FK), `tenant_id` (the voting store owner), `created_at`. Unique index on `(platform_inquiry_id, tenant_id)` to prevent double-voting. Add a denormalized `votes_count` integer on `platform_inquiries` for cheap ranking (increment/decrement in the service).

Backend:

- `app/Services/Platform/RoadmapService.php` — business logic: list published board (status ≠ `submitted` / `declined` unless owner is the requester), cast/retract vote (toggle + maintain `votes_count`), transition status (landlord only), and **fire close-the-loop notifications**.

- `app/Models/FeatureVote.php` (`HasUuid` , `BelongsToTenant`).
- FormRequests extend `BaseRequest` : `CastFeatureVoteRequest` , `UpdateRoadmapStatusRequest` .
- Resource: `RoadmapItemResource` (exposes `feature_title` , plain-language `status_label` , `votes_count` , `has_voted` — **never** raw `status` keys or tenant UUIDs).
- Routes in `routes/admin-api.php` (landlord/tenant-admin audience): `GET /roadmap` , `POST /roadmap/{uuid}/vote` , `DELETE /roadmap/{uuid}/vote` , and landlord-only `PATCH /roadmap/{uuid}/status` .
- **Close-the-loop:** on status change, `RoadmapService` calls `app/Services/EmailService.php` with a new `EmailTemplateEnum` slug (e.g. `ROADMAP_STATUS_CHANGED`) to notify all voters. **This notification is the retention mechanism — it is not optional.** Must be **queued** (see §6.4 prerequisite).

Frontend (admin `nuxt/`):

- Nav entry “Ideas & Roadmap” in `nuxt/app/components/admin/layout/Sidebar.vue` .
- Page under `nuxt/app/pages/admin/roadmap/` with a board grouped by plain-language status columns (“Under Review”, “Planned”, “In Progress”, “Shipped”).
- Pinia store + a composable (e.g. `useRoadmap()`) wrapping all calls — **no `$fetch` in components.**
- Loading skeleton, empty state (“No ideas yet — be the first to suggest one”), and error+retry are mandatory (`CLAUDE.md §0`). One-click vote with optimistic UI (low friction per research).
- Prompt a vote whenever an owner submits a feature request (reuse the existing intake form to also surface “others want this too”).

Tests: vote toggling, dedupe (unique index), status transition authorization (only landlord), and that a voter receives the close-the-loop notification on `shipped` .

6.2 Cross-Tenant Community Template Library (Audience 1) — highest moat-per-effort

Data (Postgres, additive — default-empty guarantee):

- Add to `designer_documents` (migration): `is_published_to_community` boolean **default false**, `author_tenant_id` (the publishing tenant), `community_status` enum-style (`pending` | `approved` | `rejected` , default `pending`). With defaults, behavior is byte-for-byte identical to today.
- Reuse the `StoreBlockTemplate` pattern: a `usage_count` integer + an `incrementUsage()` -style method for popularity ranking.

Backend:

- `app/Services/Designer/CommunityTemplateService.php` — queries published+approved templates **across tenants** by explicitly bypassing the global scope:
`DesignerTemplate::withoutGlobalScope(BelongsToTenant::class)->where('is_published_to_community', true)->where('community_status', 'approved')` . Feasible *only because* `designer_documents` is one shared Postgres table — no DB switching.
- **Clone flow:** “Add to my account” copies the source `designer_json` into the cloning tenant’s own `DesignerTemplate` at account level (`store_id = null`), and calls `incrementUsage()` on the source.

The clone belongs fully to the cloning tenant (normal `BelongsToTenant` write).

- Publish/unpublish endpoints in `routes/admin-api.php` ; FormRequests extend `BaseRequest` .
- **Resource never exposes tenant UUIDs** — derive an author display name (“by {Store Name}”) per `CLAUDE.md §0` .
- **Feature-flag the whole surface** (no half-built UI in prod, `CLAUDE.md §0`).

Moderation (ship *with* it, not after — guardrail):

- `community_status` lifecycle + a **landlord review queue** (list `pending` , approve/reject). Default `pending` means nothing goes live un-reviewed.
- Author attribution = store display name only.
- **Critical silent-leak test:** assert a published+approved template round-trips into the gallery AND that an **unpublished** or **un-approved** template is *never* visible cross-tenant. This is the single most dangerous bug class here (a cross-tenant data leak), so it gets an explicit test (mirrors the silent-lie test discipline in `CLAUDE.md`).

Frontend (admin `nuxt/`): “Community Templates” nav entry; gallery (reuse the `PortfolioGalleryBlock.vue` lightbox/filter *pattern* — not the storefront component itself); “Publish to community” toggle on the owner’s own templates; landlord moderation queue page. Loading/empty/error states mandatory.

6.3 Phase-2 & Phase-3 surfaces

- **In-product Changelog / Announcement feed (Audience 1):** every roadmap item that flips to `shipped` auto-posts to an in-app changelog the owner sees. Cheap, closes the loop *visibly*, and bridges to webinar/event invites (reuse `CustomerSegmentService` / `EmailCampaign` once a **landlord-side audience layer** exists — today those segment *end-customers*, not store owners, so a parallel landlord audience resolver is needed).
- **Peer Q&A / comments on Community Templates (Audience 1):** layer comments onto the template library *first* — lowest-risk forum surface. Seeds peer support (~73% deflection) without a full forum, and feeds the absent support spine. Add an “**Expert**”/“**Ambassador**” **badge** derived from high `usage_count` + adoption.
- **Benchmarking moat (Audience 1, longest horizon):** an anonymized cross-tenant metrics service computed **nightly via a scheduled command** in `routes/console.php` into an aggregate table — e.g. “your avg quote turnaround vs network median.” **Never expose another tenant’s raw data**; only network aggregates. Hardest-to-copy data network effect; build only after Phase-1/2 participation is proven.

6.4 Cross-cutting prerequisites

- **Move `QUEUE` off `sync` to Redis in prod** before any notification-driven loop ships (vote notifications, changelog digests must be queued).
- **Keep audiences strictly separated.** Roadmap / template-community / benchmarking = Audience 1. `UgcGalleryBlock` / `PortfolioGalleryBlock` stay Audience 2 storefront tools.

- **Defer a standalone external forum** (Discord/Slack/Circle). Communities need 12–18 months of genuine value before payoff — the in-product primitives above deliver retention signal faster. If a chat community is wanted near-term, run it **off-platform (Discord) as a zero-build experiment** to validate demand before any in-product spend.

7. Phased roadmap with effort sizing

Phase	Scope	Audience	Effort	Why this order
P0 / Phase 1a	Public Roadmap + Feature Voting — extend <code>platform_inquiries</code> with <code>status</code> ; add <code>feature_votes</code> + <code>votes_count</code> ; <code>RoadmapService</code> ; board page; close-the-loop email .	1	S–M (weeks) — reuses existing model, intake form, admin shell.	Highest ROI-per-effort retention primitive; data model half-exists.
P0 / Phase 1b (parallel)	Cross-Tenant Community Template Library — additive flags on <code>designer_documents</code> ; <code>CommunityTemplateService</code> (scope bypass); clone-to-account; <code>usage_count</code> ranking; moderation queue + silent-leak test ; feature flag.	1	M (weeks) — one shared table, copy <code>StoreBlockTemplate</code> pattern.	Biggest moat lever; maps to how print shops already work.
P1 / Phase 2	In-product Changelog (auto-post from roadmap <code>shipped</code>) + Peer Q&A/comments on templates + Ambassador badge . Landlord-side audience resolver for invites.	1	M	Closes the loop visibly; seeds peer support without a full forum.
P2 / Phase 3	Benchmarking — nightly aggregate command (<code>routes/console.php</code>) → anonymized network medians.	1	L (months)	Hardest-to-copy data moat; only after participation proven.
Always-deferred	Standalone external forum (Discord/Slack/Circle) as in-product build.	1	—	12–18 mo payoff; validate off-platform first.
Prereq (before any P0 notification)	<code>QUEUE</code> → Redis in prod.	infra	S	Notifications are the retention mechanism; must be queued.

8. Success metrics to track

Phase 1 — Roadmap voting

- % of active store-owner tenants who cast ≥ 1 vote (participation; target lift vs. account-gated baseline).
- Median time-to-first-status-change on a submitted idea (acknowledgement speed).
- **Close-the-loop notification open rate** on `shipped` / `planned` transitions.
- Churn / 90-day retention of voters vs. non-voters (the core hypothesis).

Phase 1 — Community Templates

• templates published to community; # approved (moderation throughput).

- Total `usage_count` (clones) across the gallery; clones-per-publisher.
- % of tenants who cloned ≥ 1 community template; retention of cloners vs. non-cloners.
- **Zero cross-tenant leak incidents** (the silent-leak test stays green in CI).

Phase 2 — Changelog & Peer Q&A

- Changelog view rate per shipped item; click-through to the related feature.

• peer answers per template question; estimated ticket deflection (proxy for the absent support spine).

• of Ambassador-badge holders and their answer/adoption share.

Phase 3 — Benchmarking

- % of tenants who view their benchmark $\geq 1\times$ /month; correlation with retention/expansion.

North-star: NRR and logo retention of community-active tenants vs. inactive — community is a retention/NRR play, and that delta is the proof it works.

9. References (file paths)

- `app/Models/DesignerTemplate.php`
- `app/Http/Controllers/Api/Designer/AccountDesignerTemplateController.php`
- `app/Http/Controllers/Api/Store/DesignerTemplateController.php`
- `app/Http/Resources/Api/Storefront/DesignerTemplateResource.php`
- `database/migrations/2026_03_16_120000_create_designer_documents_table.php`
- `app/Models/PlatformInquiry.php`

- `database/migrations/2025_07_13_093009_create_platform_inquiries_table.php`
- `app/Http/Controllers/Api/Admin/Platform/PlatformInquiryController.php`
- `nuxt/app/pages/admin/inquiries/index.vue`
- `app/Models/Store/StoreBlockTemplate.php`
- `frontstore/app/components/storefront/blocks/UgcGalleryBlock.vue`
- `frontstore/app/components/storefront/blocks/PortfolioGalleryBlock.vue`
- `nuxt/app/config/blockTypes.ts`
- `app/Services/EmailCampaign/CustomerSegmentService.php`
- `app/Services/Onboarding/StoreOnboardingService.php`
- `app/Services/EmailService.php`
- `app/Enums/EmailTemplateEnum.php`
- `routes/admin-api.php`
- `routes/console.php`
- `nuxt/app/components/admin/layout/Sidebar.vue`

Customer Success & Onboarding for Non-Technical Store Owners

Source: [readme/research/retention-lifecycle/RETENTION_CUSTOMER_SUCCESS_ONBOARDING_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about how a new print shop owner gets from “just signed up” to “my store is actually working and taking orders.” It maps what the platform already does to help owners get started, what is missing, and a step-by-step plan to fix the gaps. Getting this right is critical because most businesses that leave a software product do so within the first 90 days — usually because they never reached a real “it works!” moment.

The main parts explained simply:

- **Setup checklist** — A 5-step guide already built into the admin dashboard (“Upload logo,” “Connect payment,” etc.). It works, but its progress is stored only in the user’s browser, so it resets on a different device and the platform owner can’t see who is stuck.
- **Sample products and demo catalog** — When a store is created, some ready-made products are added automatically so the owner doesn’t start with a blank store. This is a good foundation.
- **The “first order” moment** — Research shows that the real sign a new owner will stay is when they see their first real order arrive. The current checklist marks someone as “done” before they ever reach that moment. This needs to change.
- **Test order path** — Right now there is no way for an owner to try placing an order themselves before a real customer does. Adding this would let owners see the full experience and build confidence fast.
- **Server-side progress tracking** — Saving setup progress to the database instead of the browser. This makes progress visible to all staff on the account and lets the platform owner see who is on track and who is struggling.
- **Landlord visibility and nudge emails** — The platform owner (the SaaS) currently flies blind: they can’t see which stores are active or stuck. Automated emails on Day 3 and Day 7 to stores that haven’t finished setup would provide the minimum safety net.
- **In-app help and guidance** — There is no built-in help, no short walkthrough, and no plain-language explainers for things like “what is a payment gateway?” Non-technical owners need these to get unstuck on their own.
- **Tenant health score** — A simple score (Healthy / Stable / At-Risk) based on login activity, setup progress, and orders, used to spot struggling stores early before they leave.
- **Action Center integration** — An existing “things that need attention” feed on the dashboard. Unfinished setup can be wired into this so the reminder stays visible until the store is actually live.

What to do with this: Start by saving setup progress to the database (not just the browser) and adding a “first real order received” step to the checklist — these two changes unlock everything else. Then add a way for owners to place a test order themselves, so they can reach that confidence moment without waiting on a real customer.

TL;DR

Print-Flow-360 already ships a genuinely good *first half* of new-tenant onboarding: a real, API-verified 5-step “Get your store ready” checklist on the dashboard, a seeded demo catalog so a store can transact on day one, and robust seeding services for roles/automations/CMS/shipping/payments. What it lacks is the *second half* — the part that actually drives retention. Onboarding progress lives only in browser `localStorage`, so it resets per device and is **completely invisible to the platform/landlord**; the “definition of done” is setup completion rather than the activation event that predicts retention (**the first real order**); there is no place-a-test-order path to reach that aha-moment without waiting on a real customer; there is no in-app help, tour, or contextual explainer for non-technical owners; and there is no tenant health score or proactive nudge sequence beyond a single trial-expiry email. This doc maps what exists, the gaps, the benchmarks, and a concrete P0 → P3 build plan that follows the codebase’s UUID / `BelongsToTenant` / controller → service → resource / composable conventions.

1. Why this matters for a non-technical-store-owner print SaaS

The primary users of Print-Flow-360 tenant admin are **non-technical print-shop owners** (see `CLAUDE.md` `SO`). They don’t read docs, they navigate by intuition, and they judge the product in the first session. For a SaaS aimed at this audience, onboarding *is* the product experience that decides whether they stay:

- **70% of B2B SaaS churn happens in the first 90 days** (ProductLed). The single biggest lever on retention is shortening **time-to-first-value (TTV)** — getting the owner to a real “this works for my business” moment fast.
 - Non-technical owners stall on exactly the steps Print-Flow’s checklist surfaces — “Connect a payment gateway,” “Configure shipping rates” — because they don’t know what those things *are*, not because the UI is broken. They need explainers and guidance, not just links.
 - For this audience the aha-moment is concrete and emotional: **seeing an order land in their admin**. A checklist that turns all-green but never produces that moment will still churn (Candu/Userpilot 2025).
 - The platform owner (landlord) currently flies blind: there is no way to see which tenants activated, which stalled, or which are at risk — so there is no way to run proactive customer success at all.
-

2. The two audiences — and which this doc is about

Print-Flow-360 has **two distinct onboarding/CS audiences**. They are routinely confused; keep them separate.

	Audience 1 — Platform → Tenant	Audience 2 — Store owner → End-customer
Who	The landlord/SaaS helping a new print-shop owner activate their store	The store owner engaging <i>their</i> retail customers
Surface	Admin dashboard (<code>nuxt/</code>), landlord/super-admin area, tenant lifecycle emails	Storefront (<code>frontstore/</code>), customer notifications, the 5 seeded automations
Existing assets	SetupChecklist, onboarding seeders, trial-expiry email	5 default automations seeded per tenant, end-customer emails
This doc	✅ PRIMARY FOCUS	❌ out of scope (covered by storefront retention roadmap)

This doc is about Audience 1. A key gap below is that the 5 default automations seeded for a tenant serve *their end-customers* (Audience 2) — there is **no welcome/education drip to the tenant** (Audience 1) beyond one account-created email and the trial-expiry email.

3. What already exists in the codebase

Note: an earlier grounding map claimed there was no guided setup; **that was stale/wrong**. Verified state below.

3.1 Guided setup checklist (built and wired)

- Component: `nuxt/app/components/dashboard/SetupChecklist.vue`
- Logic/composable: `nuxt/app/composables/useSetupChecklist.ts`
- Mounted on the dashboard: `nuxt/app/pages/index.vue:30` (`<DashboardSetupChecklist/>`)

It is a real “Get your store ready” checklist with: a 5-step list, a progress bar (`completedCount/requiredCount` + %), a dismiss button, a completion congrats banner, and a per-step loading state (meets `CLAUDE.md §0` loading/empty/error rules).

The 5 steps are launch-critical and verified against real API endpoints — not fake checkmarks:

Step	Verified via	Notes
1. Upload store logo	<code>store.logo_url</code>	
2. Connect a payment gateway	<code>GET /stores/{id}/integrations?category=payment</code> (<code>is_active</code> && <code>is_configured</code>)	
3. Add your first product	<code>GET /products</code> total > 0	
4. Configure shipping rates	<code>GET /stores/{id}/shipping-rates</code> length > 0	
5. Preview your storefront	—	marked optional

Steps resolve the store dynamically and deep-link to the right config page with plain-language labels (e.g. “Set Up Payment Gateway”).

3.2 Demo / sample catalog (seeded, well-handled in UI)

- `app/Services/Onboarding/StoreOnboardingService.php:858` seeds **4 demo products** with `is_demo=true`, plus 4 categories, a `WELCOME10` coupon, Standard (\$5) + Pickup shipping, and an **offline Cheque/Bank-Transfer payment method** — so a new store can transact on day one.
- The products list surfaces this responsibly: a sample-count banner + a “**Remove samples**” action with a proper consequence-stating confirm dialog (`nuxt/app/pages/products/index.vue:34, 49, 400-404`) and a per-row “Sample” badge (`:129, 326`).

This is a strong day-one-value foundation and a pattern to reuse (see P0 test-order recommendation).

3.3 Seeding-based onboarding services (robust)

- `app/Services/Onboarding/TenantOnboardingService.php` — roles, 5 default automations, statuses, email templates.
- `app/Services/Onboarding/StoreOnboardingService.php` — settings, 8 system pages, 11-block homepage CMS, footer, nav, SEO, shipping, offline payment.
- Documented in `readme/ONBOARDING.md`. Entry commands: `app:new-tenant-registered` / `app:new-store-created`.

3.4 Existing proactive nudges & lifecycle emails

- `app/Jobs/System/NotifyTrialExpiringJob.php` — scheduled `dailyAt('08:00')` in `routes/console.php`; emails the tenant admin at the configured window + 1-day mark. **This is the only proactive onboarding/retention nudge today.**
- Tenant lifecycle emails: `NotifyTenantApprovedJob` / `Blocked` / `Deactivated` / `Reactivated` via `app/Services/Admin/AccountStatusService.php`.

3.5 Action Center (reusable proactive-alert surface)

- `app/Services/ActionCenter/` produces a computed “what needs action now” feed (e.g. `app/Services/ActionCenter/Rules/FailedPaymentsRule.php`), surfaced on the dashboard via `DashboardActionCenterWidget.vue`. This is an ideal host for onboarding/CS nudges (see P2 `OnboardingIncompleteRule`).

4. Gaps

1. **No server-side activation/onboarding tracking (the single biggest gap).** Grep confirms there are no `onboarding_progress` / `setup_progress` / `activated_at` / `first_order_at` / `onboarding_completed` columns on any model or migration. The checklist’s entire state lives in browser `localStorage` (`pf360_setup_checklist_v1`, `useSetupChecklist.ts:10`). Consequences:

- (a) progress resets per browser/device and is invisible to a second staff user; (b) the landlord has **zero visibility** into which tenants activated — no proactive CS, no funnel metric, no at-risk detection.
2. **The aha-moment is wrong.** Per prior conversion-funnel research the North Star is “store live + **first order** in 7 days,” but the checklist’s definition of done is purely *setup* completion (logo/payment/product/shipping). It never tracks the activation event that actually predicts retention — the first real customer order. There is no “first order received” milestone, celebration, or signal anywhere.
 3. **No demo/sandbox mode — and no place-a-test-order path.** Only `is_demo` *sample rows* are seeded into the real store. There is no isolated “try with fake data” experience and, crucially, no way for the owner to run their own checkout/order flow before a real customer does. Today the aha-moment can only happen via a real customer.
 4. **No in-app guidance / product tour / help center.** Grep for `tour` / `driver.js` / `shepherd` / `intro.js` / `help-center` / `?-help-menu` found nothing; no `*help*` or `*support*` page under `nuxt/app/pages`. `readme/qa_section_support.md` already flags absent in-app help / Help-menu / guided tour as High gaps. Non-technical owners get a checklist but no contextual “what is a payment gateway?” explainers, no first-run tour, no searchable help.
 5. **No time-to-value instrumentation.** Nothing records when a tenant registered vs. completed each step vs. first order — so TTV (the core onboarding KPI; PLG target <24h) cannot be measured, and onboarding-completion-rate (industry avg ~19%) is unknowable.
 6. **No health-score / at-risk model for tenants.** No `RiskScoringService`, no tenant health/engagement score. The only churn signal is binary subscription status + the trial-expiry email — no early warning to trigger proactive outreach to a stalled new tenant.
 7. **Checklist hard-expires at 30 days** regardless of whether the store went live (`useSetupChecklist.ts` `EXPIRE_DAYS=30`). A tenant who stalls on day 5 *loses their guidance* on day 30 — the opposite of proactive CS.
 8. **Empty-state nudges aren’t tied to onboarding.** Module list pages have good empty states (products especially), but there’s no cross-app contextual “you haven’t connected a payment gateway yet → here’s why it matters” nudge outside the dashboard checklist. Once the checklist is dismissed, guidance disappears everywhere.
 9. **No welcome/education drip to the tenant (Audience 1).** The 5 seeded automations serve *end-customers* (Audience 2). For the tenant there is only a single account-created email (`SendNewUserEmailJob`) and the trial-expiry email — no Day 1 / Day 3 / Day 7 finish-setup sequence, and the landlord-side email engine has no time-delayed multi-step journeys to build one on.

5. Best practices & benchmarks (cited)

- **Define ONE observable activation event before designing onboarding.** Activation (real value reached) beats checklist completion — a user who finishes every step without a value moment still

churns. — Candu, 2025 (<https://www.candu.ai/blog/best-saas-onboarding-examples-checklist-practices-for-2025>)

- **Keep the checklist to 3–5 steps centered on core value.** 15-step tours convert <20%. Checklists work via the Zeigarnik effect (open loops) + visible progress. — Candu / Flowjam 2025 (<https://www.flowjam.com/blog/saas-onboarding-best-practices-2025-guide-checklist>)
- **Benchmarks:** onboarding completion averages ~19.2% (median 10.1%); >40% is good, >60% exceptional. TTV target for PLG is <24h, <1h strong, <5min world-class. — Candu, 2025
- **Show non-technical users only what they need to activate; defer non-essentials; personalize by role.** — insaim.design / Candu 2025 (<https://www.insaim.design/blog/saas-onboarding-best-practices-for-2025-examples>)
- **70% of B2B SaaS churn happens in the first 90 days** — improving time-to-first-value has the most direct retention impact; instrument and intervene early. — ProductLed (<https://productled.com/blog/saas-low-touch-customer-onboarding>)
- **Use a mixed model:** self-serve onboarding for everyone, automated proactive outreach when a user gets stuck, human help reserved for high-value/at-risk accounts. — Gainsight 2026 (<https://www.gainsight.com/essential-guide/customer-success/>); LowChurn 2025 (<https://www.lowchurn.com/blog/customer-success-strategies>)
- **Health scores combine product-usage + engagement + sentiment + business signals** to flag Healthy / Stable / At-Risk weeks before churn. — EverAfter 2025 (<https://www.everafter.ai/glossary/customer-health-score>); Accoil (<https://www.accoil.com/blog/customer-health-score>)
- **Self-serve education** (searchable help center + templates/examples) lets non-technical users unblock themselves at scale and is a prerequisite for low-touch CS. — Userpilot (<https://userpilot.com/blog/customer-success-tools/>)

6. Recommended architecture for THIS codebase

All new work must follow `CLAUDE.md §5` invariants: `HasUuid` , `BelongsToTenant` , controller → `FormRequest` → service → resource → `successResponse()` , business logic only in `app/Services/{Module}/` , every `$fetch` wrapped in a composable, Pinia for state, tests for any service method with business logic. Database is **PostgreSQL** — write Postgres-compatible schema.

6.1 Server-side onboarding/activation record (foundation for everything)

New migration + model. One row per store (tenant-scoped). Postgres notes inline.

```

Table: store_onboarding_progress (tenant-scoped; HasUuid + BelongsToTenant)
- id          bigint PK
- uuid        uuid (HasUuid)
- tenant_id   uuid (BelongsToTenant — do NOT add manual where('tenant_id'))
- store_id    uuid FK -> stores
- registered_at timestampz NULL
- logo_done_at timestampz NULL
- payment_done_at timestampz NULL
- product_done_at timestampz NULL
- shipping_done_at timestampz NULL
- storefront_previewed_at timestampz NULL
- first_order_at timestampz NULL -- the activation event
- activated_at timestampz NULL -- store live AND first order
- dismissed_at timestampz NULL
- timestamps
unique(store_id)

```

Use nullable `timestampz` per step (a timestamp answers both “is it done?” and “when?”, enabling TTV measurement) rather than booleans.

- **Model:** `app/Models/StoreOnboardingProgress.php` — `use HasUuid, BelongsToTenant;`
- **Service:** `app/Services/Onboarding/OnboardingProgressService.php`
 - `computeProgress(Store $store): array` — **business logic** ⇒ **requires a test (CLAUDE.md §5)**. Reuse the exact same checks `useSetupChecklist.ts` already performs server-side: logo via `store.logo_url`, payment via the payment integrations active+configured check, product count > 0, shipping rates length > 0. Persist each `*_done_at` the first time its check passes.
 - `recordFirstOrder(Store $store): void` — sets `first_order_at` (idempotent) and, when setup is also complete, `activated_at`. Called from `app/Services/OrderService.php` at the same ~25 automation-dispatch sites that already fire on order creation.
- **Controller/route:** `GET /onboarding/progress` (tenant store-api) → `OnboardingResource`.
- **Frontend:** rewrite `useSetupChecklist.ts` to read/write the API; keep `localStorage` only as a fast-paint cache. This fixes per-device reset, makes progress visible to all staff, and unlocks landlord visibility.

6.2 First-order milestone + aha-moment celebration

- Add a 6th, **non-dismissible** step “Receive your first order” to `SetupChecklist.vue`, driven by `first_order_at`.
- On first order, fire a celebration: toast (`showSuccess()`) + a dashboard banner in plain language (“You got your first order! 🎉”). Detect the transition client-side from the progress API, or push via the existing notification mechanism.
- Stop hard-expiring guidance at 30 days while the store isn’t live (see 6.7).

6.3 “Place a test order” path (cheapest TTV win)

- Reuse the seeded demo product + `WELCOME10` + the offline payment method so the owner runs their **own** checkout end-to-end and sees an order land in admin.

- Extend the existing `is_demo` concept to orders so the test order is clearly labeled and removable alongside “Remove samples” (mirror `nuxt/app/pages/products/index.vue:34, 49, 400-404`).
- Surface it as a checklist sub-action (“Try placing a test order”) that deep-links into the storefront checkout pre-loaded with the demo product.

6.4 Landlord-side onboarding visibility + nudges (Audience 1)

- Surface the new activation record in the super-admin/landlord area as a **funnel** (registered → setup complete → first order) and a **per-tenant status** (Activated / Stalled / At-Risk). New admin components under `nuxt/` (landlord area).
- Add a scheduled job in `routes/console.php` alongside `NotifyTrialExpiringJob` — `app/Jobs/System/NotifyStalledOnboardingJob.php` — that emails tenants who registered but haven’t completed setup / placed a first order by **Day 3 / Day 7**, using `app/Services/EmailService.php` + a new `EmailTemplateEnum` slug `ONBOARDING_NUDGE` (`app/Enums/EmailTemplateEnum.php`). This is the minimum proactive-CS layer; today only the trial-expiry job exists.

6.5 In-app Help / guidance surface

- (a) A persistent **Help** entry in the admin layout (`nuxt/app/layouts/store-management.vue`) opening a slide-over with searchable articles + links into the relevant `readme/` docs.
- (b) Contextual one-line explainers on each checklist step honoring `$0` plain language: e.g. “A payment gateway lets customers pay you online — we’ll walk you through it.”
- © A lightweight **first-run tour** (3–5 stops max: Products → Storefront → Orders). Tours >15 steps convert <20%, so stay minimal. A small dependency (driver.js) or a hand-rolled spotlight; gate behind the onboarding-progress state so it shows once.

6.6 Tenant health score (early at-risk detection)

- `app/Services/CustomerSuccess/TenantHealthService.php` — score from: days-since-registration vs steps-completed, `first_order_at` presence, login recency (`Tenant` already tracks `last_used_at` , `app/Models/Tenant.php`), order-volume trend, subscription status.
- Output Healthy / Stable / At-Risk; surface on the landlord dashboard and drive the stalled-onboarding nudge job (6.4). Combines usage + engagement + business signals per EverAfter/Accoil; targets the first-90-day churn window.

6.7 Smart escalation instead of 30-day expiry

- In `useSetupChecklist.ts` , don’t hide guidance at 30 days if the store isn’t live; **escalate** the dashboard nudge and feed the **Action Center** via a new `app/Services/ActionCenter/Rules/OnboardingIncompleteRule.php` (mirror `FailedPaymentsRule.php`) so “Finish setup to start selling” stays visible until activation. Reuses the Action Center surface already on the dashboard.

6.8 Self-serve education content

- Seed a small library of print-shop product templates/examples (business cards, flyers) the owner can one-click-add — lowers “add your first product” friction. Templates + examples are a proven low-touch activation lever (Userpilot).
- Seed a starter help-center article set for the Help slide-over (6.5).

7. Phased roadmap

Phase	Item	Effort	Why now
P0	6.1 Persist onboarding state server-side + <code>first_order_at</code> activation milestone (migration, <code>StoreOnboardingProgress</code> , <code>OnboardingProgressService</code> + test, <code>GET /onboarding/progress</code> , rewire <code>useSetupChecklist.ts</code>)	L (~3–5 d)	Fixes the single biggest gap; unblocks all landlord CS visibility and metrics
P0	6.2 Redefine “done” as store-live-AND-first-order; non-dismissible “Receive your first order” step; aha-moment celebration; stop 30-day hard expiry	S–M (~1–2 d)	Aligns onboarding with the retention-defining event
P0	6.3 “Place a test order” path reusing demo product + offline payment + <code>is_demo</code> on orders	M (~2–3 d)	Cheapest way to deliver TTV <24h without a real customer
P1	6.4 Landlord funnel + per-tenant status + <code>NotifyStalledOnboardingJob</code> (Day 3 / Day 7) + <code>ONBOARDING_NUDGE</code> template	M (~2–4 d)	Minimum proactive-CS layer; today only trial-expiry exists
P1	6.5 In-app Help slide-over + step explainers + 3–5-stop first-run tour	M (~3 d)	Non-technical owners can self-unblock; flagged High in <code>qa_section_support.md</code>
P2	6.6 <code>TenantHealthService</code> (Healthy/Stable/At-Risk) on landlord dashboard	M (~2–3 d)	Early warning for the first-90-day churn window
P2	6.7 <code>OnboardingIncompleteRule</code> in Action Center; replace hard expiry with escalation	S (~1 d)	Keeps guidance alive until activation; reuses existing surface
P3	6.8 Product-template library (one-click-add) + starter help articles	M (~2–3 d)	Low-touch activation lever; lowers first-product friction

Effort key: S ≈ ≤1 day, M ≈ 2–4 days, L ≈ 3–5 days (single engineer, includes tests).

8. Success metrics to track

Once 6.1 lands, the following become measurable for the first time:

- **Activation rate** — % of registered tenants that reach `first_order_at` (North Star: store live + first order within 7 days).
 - **Time-to-value (TTV)** — `first_order_at - registered_at`. Target <24h (PLG); track median + p90.
 - **Onboarding completion rate** — % reaching all required steps `*_done_at`. Benchmark: avg ~19%; aim >40%, exceptional >60%.
 - **Per-step drop-off** — distribution of last-completed step (which step stalls non-technical owners — likely payment or shipping).
 - **Test-order adoption** — % of tenants who place a test order before their first real order.
 - **Stalled-tenant recovery** — % of Day-3/Day-7 nudge recipients who subsequently activate.
 - **First-90-day retention** — cohort retention split by activated vs. not (validates the activation hypothesis).
 - **Tenant health distribution** — Healthy / Stable / At-Risk counts trend; correlate At-Risk → churn to tune the score.
-

9. Key file references

- `nuxt/app/components/dashboard/SetupChecklist.vue` — the checklist UI
- `nuxt/app/composables/useSetupChecklist.ts` — checklist logic (localStorage state, `EXPIRE_DAYS=30`)
- `nuxt/app/pages/index.vue` — dashboard mounting the checklist (`:30`)
- `nuxt/app/pages/products/index.vue` — sample banner + “Remove samples” pattern to mirror (`:34, 49, 129, 326, 400-404`)
- `app/Services/Onboarding/TenantOnboardingService.php` — tenant seeding (roles, automations, statuses, templates)
- `app/Services/Onboarding/StoreOnboardingService.php` — store seeding; demo products `is_demo` at `:858`
- `app/Jobs/System/NotifyTrialExpiringJob.php` — only existing proactive nudge
- `app/Services/Admin/AccountStatusService.php` — tenant lifecycle emails
- `app/Services/EmailService.php` — email engine for the new `ONBOARDING_NUDGE`
- `app/Enums/EmailTemplateEnum.php` — add `ONBOARDING_NUDGE` slug
- `app/Services/OrderService.php` — call `recordFirstOrder()` at the order-creation/automation-dispatch sites
- `app/Services/ActionCenter/Rules/FailedPaymentsRule.php` — pattern for `OnboardingIncompleteRule`
- `routes/console.php` — register `NotifyStalledOnboardingJob` alongside trial-expiry

- `app/Models/Tenant.php` — `last_used_at` login-recency signal for health score
- `nuxt/app/layouts/store-management.vue` — host for the Help entry
- `readme/ONBOARDING.md` — existing seeding documentation
- `readme/qa_section_support.md` — flags absent in-app help / tour as High gaps

Dunning & Involuntary Churn Recovery — Failed/Expired-Card Payment Recovery on SaaS Subscriptions

Source: [readme/research/retention-lifecycle/RETENTION_DUNNING_INVOLUNTARY_CHURN_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about what happens when a store owner's credit card payment fails for their Print-Flow-360 subscription — and how the platform should handle it. Right now, nothing happens: the card can quietly fail for weeks, no email is sent, and the store owner's access is never cut off. Fixing this is one of the highest-return improvements possible, because most of these failures are accidents (an expired card), not the owner wanting to leave.

The main parts explained simply:

- **Involuntary churn** — When a paying customer loses access not because they want to cancel, but because their card expired or was declined by the bank. Industry research shows this causes 20–40% of all subscription cancellations. It is recoverable revenue — the person still wants the product.
- **Two billing relationships** — The platform has two completely separate money flows: (1) store owners pay Print-Flow-360 a monthly fee, and (2) shoppers pay each store for print orders. This document is only about the first one. The two must never be mixed up in the code.
- **What already exists** — The billing engine, payment adapters (Stripe, Razorpay, etc.), the subscription database table, and the card expiry date field are all already built. The gap is everything on top: reacting to failures, sending emails, retrying charges.
- **The gaps (what is broken)** — No code ever marks a subscription as “payment failed.” No retry schedule exists. Stripe sends failure notifications but nothing reads them. The warning window before access is cut is only 1 day (industry standard is 7–14). There are no recovery emails and no “update your card” page.
- **Pre-dunning warning** — Sending a friendly email about 30 days before a card expires, before any failure even happens. The card expiry date is already stored. This single step recovers an estimated 15–22% of at-risk revenue and is the easiest improvement to ship.
- **Dunning email sequence** — A series of 4 short emails sent after a payment fails (Day 0, 3, 5, 7), each with a one-click link to update the card. Good sequences recover 50–65% of failed payments.
- **Retry schedule** — Automatically trying the charge again on Day 1, 3, 5, and 7 after failure. For Stripe, this is partly handled by Stripe itself; for other payment providers, Print-Flow-360 must do it directly.
- **Self-serve recovery page** — A simple page in the admin dashboard where a suspended store owner can update their card and immediately restore access. Plain language, no technical errors, no dead buttons.
- **Phased build plan** — P0 makes failures visible (wire Stripe notifications, fix the silent-ignore bug). P1 recovers the revenue (pre-dunning email, retry jobs, email sequence, recovery page). P2 measures results.

What to do with this: Start with P0 — right now a card can fail silently for weeks and the platform never knows. After that, the card-expiry warning email (pre-dunning) is the single fastest revenue win: the data is already stored, and one job prevents the failure entirely.

TL;DR — Print-Flow-360 charges store-owner tenants a recurring SaaS subscription (\$149/mo + \$39/user, 14-day trial), and Stripe even creates a real native recurring subscription that retries failed cards. But **nothing in the codebase reacts to a failed charge**: no subscription webhooks are consumed, the `charge_failed` status is never written by any code path, `BillingService::renew()` is never called by any scheduler, there is no card-expiry warning, no dunning email sequence, no self-serve “update your card” recovery flow, and the grace window is a token 1 day. A tenant’s card can silently expire or decline for weeks while the access middleware keeps reading `active`. Since failed payments cause 20–40% of SaaS churn and good dunning recovers 50–65% of it, this is high-leverage, mostly-additive revenue-recovery work built on substrate that already exists (the unified webhook dispatcher, the subscription models, the system-mail/cron pattern).

Status: research-only (nothing built yet). This document is a plan and a map, not a record of shipped work.

1. Why this matters for a non-technical-store-owner print SaaS

Our paying customers are **non-technical print-shop owners**. When their saved card expires (cards expire on a schedule nobody remembers) or a bank declines a renewal, three things happen today:

1. **They get no warning.** No card-expiry email, no failed-payment email. The first signal would be losing access — except that doesn’t even fire reliably (see §4).
2. **We lose revenue we would have kept.** Most of these failures are *involuntary* — the owner still wants the product, the card just lapsed. Industry data puts involuntary churn at **20–40% of all SaaS churn** (ProfitWell / State of Retention). Recovering it is pure margin: no new acquisition spend, the customer already wants to stay.
3. **It reads as “the software broke.”** A non-technical owner whose store suddenly 402s with no explanation will assume the product failed, not that their card lapsed. That is exactly the failure mode [CLAUDE.md §0](#) forbids (“a blank screen reads as broken”; “tell the user what to do next”). Recovery must be a plain-language, one-click “update your card” path — never a silent block.

Dunning is one of the highest-ROI features a subscription business can ship: typical dunning ROI is **10–15×** because it recovers revenue that already exists. For a print SaaS whose buyers are busy, non-technical, and not watching their billing, the *pre-dunning* warning (before the card even fails) is the single highest-value, lowest-effort piece — the data is already stored.

2. The two audiences — which one this doc is about

Print-Flow has **two completely separate billing relationships**, and dunning means something different for each. Getting this wrong is the #1 implementation trap here.

	Audience 1 — Platform → Tenant	Audience 2 — Store-owner → End-customer
Who pays whom	The print-shop owner (tenant) pays us the SaaS fee	A shopper pays the store for a print order
Billing engine	<code>BillingService</code> + <code>SubscriptionGatewayRegistry</code> (recurring subscription)	Order checkout, payment gateways per store
Audience scope	Landlord / system — NOT tenant-scoped	Tenant-scoped (lives inside one store's data)
Failure today	Silently ignored (this doc)	Surfaced passively via <code>FailedPaymentsRule</code> + <code>PAYMENT_FAILED</code> tenant email template
Mail path	<code>app/Jobs/System/*</code> + <code>resources/views/emails/system/*</code> (e.g. <code>NotifyTrialExpiringJob</code>)	Tenant Automation engine + tenant email templates

This document is about Audience 1 — platform-to-tenant subscription dunning. That is the gap.

The existing failed-payment touchpoints are all Audience 2 and are **the wrong audience** to reuse:

- `app/Services/ActionCenter/Rules/FailedPaymentsRule.php` — internal owner alert for a *store's* failed order payments. Tenant-scoped. Not us retaining the tenant.
- `EmailTemplateEnum::PAYMENT_FAILED` (`'payment_failed'`) — fires for a store's end-customer order payment failure. Tenant-scoped. Reusing it for SaaS dunning would leak landlord billing into a store's email system.
- The per-tenant **Automation engine** has no time-delay steps and is tenant-scoped — it cannot drive a multi-step landlord dunning sequence.

Consequence for the build: all dunning jobs and Blade views must live on the **landlord side** (`app/Jobs/System/` , `resources/views/emails/system/`), mirroring `NotifyTrialExpiringJob` . Do **not** route platform dunning through tenant-scoped machinery.

3. What already exists in the codebase

There is a surprising amount of reusable substrate — the gap is the reactive/recovery layer on top of it, not the foundation.

Billing engine (orchestration)

- `app/Services/BillingService.php` — orchestrates `subscribe()` / `renew()` / `cancel()` behind `SubscriptionGatewayInterface` . Writes `SubscriptionTransaction` rows **only on success** (no failed rows — a key gap).
- `app/Services/Billing/SubscriptionGatewayRegistry.php` — 6 adapters: Stripe, Razorpay, PayPal, AuthorizeNet, PayTM, Cheque.

- `app/Services/Billing/StripeBillingGateway.php` — the **only** adapter that creates a real native recurring Stripe Subscription (`stripe->subscriptions->create` , `interval=month` , `default_payment_method` attached). So Stripe *itself* retries cards and emits `invoice.payment_failed` / `invoice.paid` — but **nothing local consumes those events**. The other 5 gateways do a one-time charge with **no native recurring/retry** (Cheque is manual).

Status enforcement (the only active “involuntary churn” touchpoint — purely defensive)

- `app/Http/Middleware/EnsureTenantHasActiveSubscription.php` — reads a resolved status, returns **402** for `charge_failed` / `expired` / `no_subscription` . Has a grace window (`BillingSettings.grace_period_days` , **default 1**) that allows access and sets an `X-Grace-Period` header (amber banner). Grace is **computed inline** from `ends_at + grace_period_days` — there is no grace *state*. This middleware **blocks** access; it does not **recover**.
- `app/Services/Subscription/SubscriptionStatusCache.php` — resolves + caches status (`SUBSCRIPTION_MODE` local|remote, 15-min Redis cache, **fail-open** on outage). Remote mode calls central platform `POST /api/v1/internal/subscription/verify` (`routes/admin-api.php` → `SubscriptionVerifyController`).

Models & schema

- `app/Models/Subscription.php` — status enum includes `'charge_failed'` ; uses `$guarded = []` . Helper `isActiveForAccess` .
- `SubscriptionTransaction` , `SubscriptionPlanChange` , `Plan` , `BillingSettings` , `TenantPaymentProfile` .
- **PlanSeeder** — \$149/mo + \$39/user, 14-day trial.
- **tenant_payment_profiles** (migration `2026_05_25_000004_add_gateway_fields_to_tenant_payment_profiles_table.php`) — stores `expiration_date` , `last_four` , `card_brand` , `gateway_customer_id` , `gateway_payment_profile_id` . **This is the pre-dunning goldmine** — `expiration_date` is already **captured**. ⚠ `card_number` + `cvv` are stored **plaintext** — separate P0 security bug already logged; do not build on top of plaintext card data, use the gateway tokens (`gateway_customer_id`) instead.

Reusable webhook substrate (the key enabler)

- `routes/public-api.php` — `POST /api/v1/webhooks/payments/{gateway_key}` → `PaymentWebhookController` → `app/Services/Payment/PaymentEventDispatcher.php` : signature verification, **idempotent** `PaymentWebhookEvent` dedup (`gateway_key + event_id`), clean two-phase claim/dispatch. This is a clean, reusable substrate — but its `EVENT_MAP` only maps **order-payment** events and `resolveOrderId()` **requires an Order**, so subscription events are silently dropped (`ignored_unknown`).
- `app/Models/PaymentWebhookEvent.php` — idempotency ledger.

Proactive nudge pattern to mirror

- `app/Jobs/System/NotifyTrialExpiringJob.php` — scheduled `dailyAt('08:00')`, renders `resources/views/emails/system/trial-expiring.blade.php`, sent via the system mail path. **This is the exact template to copy** for card-expiry and dunning jobs.
- `app/Services/Admin/AccountStatusService.php` + `app/Jobs/System/*` — account lifecycle emails (approved/blocked/deactivated/reactivated/signup).

4. Gaps (what's missing or broken)

1. **No retry/dunning engine at all.** `BillingService::renew()` exists but **nothing calls it** — `routes/console.php` has no renewal/retry entry; nothing in `app/Console` or `app/Jobs` calls `renew()`. No recurring-charge cron, no retry schedule, no escalation.
2. **charge_failed is a dead write path.** It appears only in the enum migration (`2026_05_26_000001`) and in **read** sites (middleware, `isActiveForAccess`). **No code ever SETS** `Subscription.status = 'charge_failed'`. The middleware branch that blocks `charge_failed` tenants can therefore **never trigger from a real failure**.
3. **Subscription webhooks are not handled.** `PaymentEventDispatcher::EVENT_MAP` maps only order events (`payment.authorized/captured/failed/refunded/disputed`) and `resolveOrderId()` requires an Order. Stripe's `invoice.payment_failed`, `invoice.paid`, `customer.subscription.updated/deleted`, `invoice.upcoming` carry no `order_id` → **silently ignored**. Stripe can dunning-fail a card for weeks; the local `Subscription.status` never changes; middleware keeps reading `active`.
4. **Latent correlation bug — missing gateway_subscription_id column.** `StripeBillingGateway` reads `$subscription->gateway_subscription_id` and stores `'stripe_subscription_id'` in its result, but **no migration adds gateway_subscription_id to subscriptions**. So `cancel()` / lookups read `null`; the Stripe subscription cannot be reliably correlated to the local record. Fixing this is a prerequisite for webhook matching.
5. **No pre-dunning.** No card-expiry warning job despite `tenant_payment_profiles.expiration_date` being available. Card silently expires → the first failed charge is the first signal.
6. **No platform → tenant dunning emails.** `resources/views/emails/system/` has `trial-expiring`, `account-*`, `signup` — no `payment-failed` / `past-due` / `final-notice`. No multi-step nurture. (The tenant Automation engine is the wrong audience and has no delay steps.)
7. **No formal account state machine.** States are ad-hoc strings spread across `Subscription.status`, `Tenant`'s `AccountStatusEnum`, and the middleware's inline branches. No `active → past_due → suspended → cancelled` lifecycle, no soft-vs-hard suspension, no grace `state` column — grace is recomputed on the fly with a **1-day default**, far below the 7–14 day norm.
8. **No self-serve recovery flow.** Middleware whitelists billing routes, but there is no recovery landing page, no Stripe SetupIntent / Billing-Portal handoff, no one-click “update card” link anywhere.

9. **No recovery instrumentation.** No recovered/failed metrics, no voluntary-vs-involuntary split.
`SubscriptionTransaction` rows with `status='failed'` are **never written** (only `'success'`).
10. **Multi-gateway reality unaddressed.** Only Stripe is natively recurring.
Razorpay/PayPal/AuthorizeNet/PayTM do one-time charges with no native retry → for those, *even passive Stripe-style retries don't exist*. A unified retry policy must live **in Print-Flow**, not be delegated per-gateway.
11. **Reliability risk:** `QUEUE=sync` **in prod** (per memory). Any retry/dunning job needs a Redis queue first, or scheduled charges run inline and can wedge web requests.
-

5. Best practices & benchmarks (with sources)

- **Involuntary churn is huge.** Failed payments cause **20–40% of all SaaS churn**; dunning ROI is typically **10–15×**. (ProfitWell/State of Retention; baremetrics.com/blog/dunning-management; getmonetizely.com)
- **Retry schedule.** A fixed **Day 1 / 3 / 5 / 7** schedule recovers **~58%** of failed payments with *no* communication; ~5 retries is the sweet spot. Smart/ML retries (Stripe default ~4 attempts over 2–3 weeks, configurable up to “8 tries within 2 weeks”) add 10–15% by timing to timezone, historical success, and decline code. (slickerhq.com; payproglobal.com; churndog.com; docs.stripe.com/billing/revenue-recovery/smart-retries)
- **Realistic recovery rate.** Good dunning recovers **50–70%** (AI/optimized 65–85%); vendor “90%” claims are inflated — FlexPay real-world data shows 25–52%. **Plan for ~50–65%**. (slickerhq.com/.../reality-check; ustechautomations.com)
- **Pre-dunning wins.** Warn **~30 days before card expiry** and prompt an update — higher action rate than any post-failure email; recovers an additional **15–22%** of at-risk revenue by preventing the failure entirely. (sequenzy.com; ustechautomations.com)
- **Three-part recovery system.** (1) processor smart-retry timing, (2) a **6–7 email dunning sequence over ~30 days**, each with a one-click billing-update link, (3) escalation that **pauses rather than cancels** high-value accounts + personal outreach for the largest. (ustechautomations.com; sequenzy.com)
- **Stripe owns the charge, you own the rest.** `invoice.payment_failed` starts dunning; Stripe Smart Retries handles attempts but **explicitly does NOT** handle comms sequencing, grace periods, fallback payment methods, or engagement — the app must own those. Restrict features for `past_due` via webhook to create urgency. (docs.stripe.com/billing/subscriptions/webhooks)
- **Grace window 7–14 days** (Print-Flow's 1-day default is far too short). Use a distinct `past_due` / `grace` state with **soft suspension** (read-only + banner) before **hard suspension**. (docs.stripe.com/invoicing/automatic-collection; viprasol.com)
- **Canonical state machine:** `active` → `past_due` (retrying + dunning) → `grace/paused (soft)` → `unpaid/suspended (hard)` → `canceled`, with recovery edges back to `active` on successful card update/charge. (baremetrics.com)

- **Tailor by decline type:** hard declines (lost/stolen/closed) → stop retrying, go straight to “update card”; soft declines (insufficient funds) → time-spaced retries aligned to pay cycles (1st/15th in US). (payproglobal.com; churndog.com)

6. Recommended architecture for THIS codebase

Design principles, all per [CLAUDE.md](#): **landlord-side** plumbing (not tenant Automation), **controller** → **service** → **resource** layering, business logic in `app/Services/{Module}/`, **audit-log** every state transition, **UUID** routes/URLs, Postgres-compatible migrations, plain-language UX, no dead buttons.

6.1 Data model changes (Postgres)

Change	Table	Notes
Add <code>gateway_subscription_id</code> (string, nullable)	<code>subscriptions</code>	New migration. Fixes the latent <code>StripeBillingGateway</code> null-read (gap #4). Prerequisite for webhook correlation + <code>cancel()</code> . Persist in <code>BillingService::activateSubscription()</code> from <code>gatewayResult['gateway_transaction_id']</code> . <code>\$guarded=[]</code> so no <code>\$fillable</code> edit needed.
Add explicit lifecycle state	<code>subscriptions</code>	Either widen the existing <code>status</code> enum (Postgres: drop <code>{table}_status_check</code> CHECK constraint, <code>->change()</code> to string, re-add) to include <code>past_due</code> , <code>grace</code> , <code>suspended</code> , or add a dedicated <code>lifecycle_state</code> column. Prefer reusing/widening <code>status</code> to avoid two sources of truth.
Add <code>past_due_since</code> (timestamp, nullable)	<code>subscriptions</code>	Anchors the Day 1/3/5/7 retry clock and grace expiry.
Add <code>recovered_at</code> (timestamp, nullable)	<code>subscriptions</code>	Recovery-rate instrumentation (metric \$8).
Add <code>grace_period_days</code> already exists	<code>billing_settings</code>	Change default 1 → configurable 7–14; surface in admin billing settings UI.
Write <code>status='failed'</code> / <code>'retried'</code> rows	<code>subscription_transactions</code>	Currently only <code>'success'</code> is written. Needed for the funnel metric.

Do **not** add new card columns — use the existing `gateway_customer_id` / `gateway_payment_profile_id` tokens. Never build on the plaintext `card_number` / `cvv` (separate P0).

6.2 New services (`app/Services/Subscription/`)

SubscriptionStateService — the single owner of lifecycle transitions. Methods like `markPastDue($subscription, $declineCode)` , `enterGrace()` , `suspend(soft|hard)` , `recover()` , `cancel()` . Each transition must:

1. Update `Subscription.status` + `past_due_since` / `recovered_at` .
2. Invalidate **SubscriptionStatusCache** (15-min Redis cache, else stale 402/active).
3. **Write an audit log** (CLAUDE.md significant-action rule: who/what before → after/when).
4. Emit a domain event (e.g. `SubscriptionEnteredPastDue`) that the dunning jobs + Action Center subscribe to.

Distinguish **soft suspension** (read-only + recovery banner; middleware allows `GET` + billing routes) from **hard suspension** (full 402). `EnsureTenantHasActiveSubscription` should read these explicit states instead of recomputing grace inline.

SubscriptionDunningService — drives the retry policy (Print-Flow-owned, not per-gateway). Knows the schedule (Day 1/3/5/7 + optional 14), the per-attempt cap (~5), jitter, and the decline-type branch (hard → skip retries, jump to update-card; soft → time-spaced). Calls `BillingService::renew()` with the stored gateway profile. Gate per gateway via a **capability flag on** `SubscriptionGatewayInterface` (e.g. `isNativelyRecurring(): bool`): for Stripe, trust Stripe Smart Retries + webhooks and run no Print-Flow retries; for Razorpay/PayPal/AuthorizeNet/PayTM, Print-Flow drives the retries.

6.3 Webhook handling (extend existing dispatcher)

In `app/Services/Payment/PaymentEventDispatcher.php` :

- Extend `EVENT_MAP` with subscription-scoped events: `invoice.payment_failed` , `invoice.paid` , `customer.subscription.updated` , `customer.subscription.deleted` , `invoice.upcoming` .
- Add `resolveSubscriptionId()` alongside `resolveOrderId()` — match Stripe `invoice.subscription` / `customer` to the local `Subscription` via `subscriptions.gateway_subscription_id` (new column) or `TenantPaymentProfile.gateway_customer_id` .
- Handlers: `invoice.payment_failed` → `SubscriptionStateService::markPastDue()` + write a `SubscriptionTransaction status='failed'` ; `invoice.paid` → `recover()` + extend `renewal_at` .
- Reuse the existing **idempotent** `PaymentWebhookEvent` **dedup** and the signature-verified, no-auth-middleware webhook conventions (CLAUDE.md). Route stays `POST` `/api/v1/webhooks/payments/{gateway_key}` .

6.4 Scheduled jobs (`app/Jobs/System/` — landlord side, mirror `NotifyTrialExpiringJob`)

- **NotifyCardExpiringJob** (pre-dunning) — `dailyAt` , reads `tenant_payment_profiles.expiration_date` , emails tenants ~30 days out with an update-card link. **Highest ROI, lowest effort** — data already stored.

- **RetrySubscriptionChargeJob** — driven by a console command `billing:run-dunning` scheduled in `routes/console.php` (hourly). For each `past_due` subscription on a non-recurring gateway, attempt `BillingService::renew()` on the Day 1/3/5/7 cadence keyed off `past_due_since`.
- **Dunning email jobs** — `SendDunningEmailJob` (or discrete jobs), triggered by `SubscriptionStateService` transitions, rendering new Blade views in `resources/views/emails/system/`:
 - `dunning-1-soft` (Day 0, friendly “your payment didn’t go through”)
 - `dunning-2` (Day 3)
 - `dunning-3` (Day 5, urgency)
 - `dunning-final` (Day 7, suspension warning)
 - Each carries a **one-click recovery link** to the self-serve page. Send via the system mail path used by `NotifyTrialExpiringJob`.

6.5 Self-serve recovery flow (`nuxt/` admin)

- New billing-recovery page in `nuxt/` reachable from the 402/grace amber banner **and** every dunning email link.
- Backed by a **Stripe SetupIntent / Billing Portal session** (and equivalent re-auth for other gateways). On success: immediately retry the charge, transition state back to `active`, invalidate the cache.
- Wrap the API call in a **composable**; state in **Pinia**; never `$fetch` in a `.vue`.
- **Whitelist its route** in `EnsureTenantHasActiveSubscription::EXCLUDED_PATHS` so a suspended tenant can reach it.
- UX (`CLAUDE.md` §0): plain-language banner — “Your payment failed — update your card to avoid interruption” — reserve layout space for the async Stripe element, show a loading state in its slot, never a dead/greyed button without explanation, handle failure with retry.

6.6 Remote-mode parity

`SubscriptionVerifyController` (central, `routes/admin-api.php`) must return the new `past_due` / `grace` / `suspended` states so `SubscriptionStatusCache` + middleware behave identically in remote and local modes. Keep the three host-resolution/verify code paths consistent (per memory). Update `readme/PAYMENT_GATEWAYS.md` + `readme/DEPLOYMENT_REMOTE_SUBSCRIPTION.md` per the docs-maintenance rule.

7. Phased roadmap

P0 — Prerequisites + make failures observable (the floor)

Without this, nothing downstream can work; a card can fail for weeks unnoticed.

Item	Effort
Move <code>QUEUE=sync</code> → Redis before shipping any job (else scheduled charges run inline and wedge web requests).	S (ops)
Migration: <code>subscriptions.gateway_subscription_id</code> + persist it in <code>BillingService::activateSubscription()</code> . Fixes latent null-read.	S
Wire subscription webhooks — extend <code>PaymentEventDispatcher</code> <code>EVENT_MAP</code> + <code>resolveSubscriptionId()</code> ; <code>invoice.payment_failed</code> → mark <code>charge_failed</code> / <code>past_due</code> + write <code>SubscriptionTransaction status='failed'</code> ; <code>invoice.paid</code> → <code>active</code> . Reuse idempotent dedup.	M
SubscriptionStateService with explicit <code>active</code> → <code>past_due</code> → <code>grace</code> → <code>suspended</code> → <code>cancelled</code> , cache-invalidation + audit log; soft vs hard suspension; configurable 7–14 day grace; middleware reads explicit states.	M

Outcome of P0: the `charge_failed` middleware branch can finally fire from real failures; state is explicit and audited.

P1 — Recover the revenue

Item	Effort
NotifyCardExpiringJob (pre-dunning, ~30 days out). Highest ROI / lowest effort.	S
Print-Flow retry loop — <code>RetrySubscriptionChargeJob</code> + <code>billing:run-dunning</code> command + scheduler; Day 1/3/5/7 keyed off <code>past_due_since</code> ; cap ~5; decline-type branch; capability flag on <code>SubscriptionGatewayInterface</code> (Stripe trusts Smart Retries, others driven by Print-Flow).	M–L
Platform → tenant dunning email sequence — 4 Blade views in <code>resources/views/emails/system/</code> + jobs driven by state transitions; one-click recovery link each. Landlord-side only.	M
Self-serve recovery page (<code>nuxt/</code>) — <code>SetupIntent/Billing-Portal</code> , retry-on-success, route whitelisted, plain-language banner.	M

P2 — Measure & harden

Item	Effort
Recovery instrumentation — persist failed/retried <code>SubscriptionTransaction</code> rows; <code>recovered_at</code> ; landlord admin dashboard tile (past_due count, recovery rate, \$ at risk) Action-Center-style.	M
Remote-mode parity — <code>SubscriptionVerifyController</code> returns new states; keep verify paths in sync; update <code>readme/PAYMENT_GATEWAYS.md</code> + <code>readme/DEPLOYMENT_REMOTE_SUBSCRIPTION.md</code> .	S–M

8. Success metrics to track

- **Involuntary-churn recovery rate** = recovered tenants ÷ failed-payment tenants. Target the **50–65%** benchmark; flag if below 40%.

- **Pre-dunning save rate** — % of card-expiry-warned tenants who update before any failure (target the 15–22% at-risk-revenue lift).
 - **Recovered MRR / \$ recovered per period** (the headline number for ROI; expect 10–15× on dunning spend).
 - **Funnel counts** — `past_due` count, `grace` count, `suspended` count, `cancelled-after-dunning` count (where the sequence ended). Sourced from the new explicit states + `SubscriptionTransaction` failed/retried rows.
 - **Retry success by attempt #** (Day 1/3/5/7) and **by decline type** (hard vs soft) — tune the schedule.
 - **Voluntary vs involuntary churn split** — proves how much of total churn dunning is addressing.
 - **Time-to-recovery** (`recovered_at` - `past_due_since`).
-

9. Key file references

Area	Path
Billing orchestration	<code>app/Services/BillingService.php</code>
Stripe recurring adapter (latent bug)	<code>app/Services/Billing/StripeBillingGateway.php</code>
Gateway registry (6 adapters)	<code>app/Services/Billing/SubscriptionGatewayRegistry.php</code>
Access enforcement (only churn touchpoint)	<code>app/Http/Middleware/EnsureTenantHasActiveSubscription.php</code>
Status resolve + cache	<code>app/Services/Subscription/SubscriptionStatusCache.php</code>
Subscription model	<code>app/Models/Subscription.php</code>
Webhook dispatcher (extend this)	<code>app/Services/Payment/PaymentEventDispatcher.php</code>
Webhook controller	<code>app/Http/Controllers/Api/Payment/PaymentWebhookController.php</code>
Idempotency ledger	<code>app/Models/PaymentWebhookEvent.php</code>
Webhook route	<code>routes/public-api.php</code>
Remote verify route	<code>routes/admin-api.php</code> (→ <code>SubscriptionVerifyController</code>)
Scheduler (no renewal entry today)	<code>routes/console.php</code>
Pattern to mirror for new jobs	<code>app/Jobs/System/NotifyTrialExpiringJob.php</code>
Account lifecycle emails	<code>app/Services/Admin/AccountStatusService.php</code>
Tenant-scoped failed-payment rule (wrong audience)	<code>app/Services/ActionCenter/Rules/FailedPaymentsRule.php</code>
Tenant email enum (wrong audience)	<code>app/Enums/EmailTemplateEnum.php</code>
Subscriptions table migration	<code>database/migrations/2025_05_31_182507_create_subscriptions_table.php</code>
Status/gateway enum migration	<code>database/migrations/2026_05_26_000001_add_gateway_key_and_statuses_to_subscriptions_table.php</code>
Payment-profile gateway fields (pre-dunning data)	<code>database/migrations/2026_05_25_000004_add_gateway_fields_to_tenant_payment_profiles_table.php</code>
Trial-expiring email view (template to copy)	<code>resources/views/emails/system/trial-expiring.blade.php</code>
Docs to update (maintenance rule)	<code>readme/PAYMENT_GATEWAYS.md</code> , <code>readme/DEPLOYMENT_REMOTE_SUBSCRIPTION.md</code> , <code>readme/PAYMENTS_BILLING_FINANCIAL_CORRECTNESS.md</code>

Lifecycle & Behavioral Email (Triggered Journeys) for Print-Flow-360

Source: [readme/research/retention-lifecycle/RETENTION_LIFECYCLE_BEHAVIORAL_EMAIL_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is a plan for building automatic, timed email sequences that send themselves based on what customers do — or don't do. Right now the platform can only send one-off blast emails or a single triggered message. There is no system to send “reminder → follow-up → final offer” over a few days, and that missing piece costs stores real money every day.

The main parts explained simply:

- **Why this matters for print shops** — Print products like business cards and flyers get reordered on a regular schedule, and online print checkouts are long and complicated, so many shoppers leave halfway. Both situations are perfect for automatic follow-up emails — but today the platform sends nothing after a customer walks away.
- **Two separate email directions** — Emails going from the platform to store owners (like “your trial expires soon” or “payment failed”) need completely different plumbing than emails going from a store to its shoppers. The document keeps these two apart and plans separate solutions for each.
- **What already works** — There is already a system for sending one-shot automated emails when something happens (like an order status change), plus broadcast campaigns to groups of customers. This plan reuses those existing pieces rather than starting from scratch.
- **Abandoned cart recovery** — When a shopper adds items to their cart but leaves without buying, the platform currently does nothing. Research shows a 3-email series sent over 3 days (first within an hour) can recover a large share of these sales. This is listed as the single most valuable thing to build first.
- **Multi-step journeys** — A “journey” is a series of emails sent over days with waits in between, that stops automatically if the customer takes action (like placing an order). Today there is zero ability to do this. The plan introduces three new database records to track: the journey definition, its steps, and which customers are currently in it.
- **Post-purchase emails** — After an order is delivered, the platform currently goes silent. The plan adds automatic emails asking for a review, gathering feedback, and reminding the customer to reorder when their supply runs low.
- **Frequency cap (spam guard)** — Without a limit, three separate parts of the system (campaigns, automations, journeys) could each email the same customer with no coordination. The plan adds a shared gate that enforces a monthly per-person limit and quietly stops emailing people who have not opened anything in 60–90 days. Order and payment emails are always exempt from this limit.
- **Platform-to-owner lifecycle** — Store owners on a trial or with a failed payment currently get almost no automatic outreach. The plan adds welcome sequences, setup nudges, payment-failure retry emails (dunning), and win-back messages for lapsed owners.
- **Phased build order** — Fix the background job queue first (a technical blocker that makes timed delays impossible), then build the journey engine with abandoned-cart recovery, then add the safety cap and post-purchase emails, then build the admin interface for store owners to manage journeys.

What to do with this: The first priority is the abandoned-cart recovery sequence — it is the highest-revenue gap and uses data that already exists in the system. Before building anything, the background

job queue must be switched from “sync” mode to a real queue (Redis), because timed email delays are physically impossible without it.

TL;DR — Print-Flow-360 has strong *broadcast* email (one-shot campaigns with RFM-style segmentation) and a *single-shot* event → action automation engine, but it has **no journey/drip engine** at all: no multi-step sequences, no time-delay steps, no per-recipient enrolment state, and no behavioral lifecycle triggers (abandoned cart, post-delivery, win-back, trial activation). The single highest-ROI gap is abandoned-cart recovery (industry: ~76% of automation revenue). The recommended path is to **extend the existing automation substrate** (reuse `ActionExecutor.php` + the `[token]` parser) into a new tenant-scoped `Journey` / `JourneyStep` / `JourneyEnrolment` model trio, add lifecycle triggers, add a `journeys:tick` cron, and ship abandoned-cart recovery as the flagship journey — gated behind a global frequency-cap / suppression layer so the three independent senders can’t collectively spam a buyer.

Status: research-only (nothing built yet). This document is an implementation map for a later engineer. No code, migrations, or seeders have been created.

1. Why this matters for a non-technical-store-owner print SaaS

Print-Flow-360’s primary users are non-technical print-shop owners who do not run marketing-ops the way a SaaS growth team would. They will never hand-build a 5-email drip in a journey canvas. The product’s job is to ship **opinionated, pre-seeded, opt-in default journeys** that recover revenue and retain customers automatically — the owner flips one toggle, not configures a state machine.

Two things make this acute for a print SaaS specifically:

1. **Print is a consumable-reorder business.** Business cards, flyers, letterheads, banners, and stationery reorder on predictable cycles. A reorder-reminder timed to a consumable is free revenue that today nobody collects — the order conversation literally ends at `ORDER_SHIPPED` (see Gaps).
2. **Cart abandonment is structural in custom-print checkout.** Configuring a product (size, stock, finish, uploading artwork, proofing) is high-friction; shoppers bail mid-flow constantly. Cart rows already accumulate in `app/Models/Cart.php` and are **never acted on**. Abandoned-cart recovery is the single highest-leverage automation in e-commerce and it is entirely absent here.

Because the audience is non-technical, every journey must obey `CLAUDE.md` §0: plain-language delay labels (“Wait 1 day”, not `delay_minutes: 1440`), human empty/loading/error states, opt-in defaults seeded **disabled** so nobody is surprised by sends, and confirmation-with-consequences on enable (“This will email customers who abandon a cart, up to 3 times over 3 days.”).

2. The two audiences — and which applies

Lifecycle email in this platform splits cleanly into two audiences that **cannot share plumbing**:

Audience	Direction	Who sends → who receives	Tenant context	Existing engine
Audience 1	Platform → Tenant (landlord-side)	Print-Flow-360 → store owner	Landlord / central DB. Tenant-scoped engines cannot serve this.	Only <code>NotifyTrialExpiringJob</code> + a handful of <code>app/Jobs/System/*</code> one-shots driven by <code>AccountStatusService</code>
Audience 2	Tenant → End-customer (store-side)	A store → its shoppers	Tenant-scoped (<code>BelongsToTenant</code> , host-resolved)	Broadcast campaigns + single-shot Automation engine

Both apply, and both have journey gaps — but they need separate engines. The tenant `Automation / Journey` engine is `BelongsToTenant` and runs inside tenant context; it physically **cannot** enrol or email a store owner (Audience 1). Audience-1 lifecycle (welcome/activation, go-live nudges, dunning, post-trial win-back) must be built as **landlord-side scheduled System jobs** alongside `NotifyTrialExpiringJob` — not bolted onto the tenant journey engine.

A third, separate concern is **transactional email** (order placed, payment failed, shipped). That is `EmailService` + `EmailTemplateEnum` and must stay **exempt from frequency capping** — never throttle a payment-failed or order-shipped mail.

3. What already exists in the codebase

3.1 Broadcast campaign engine (Audience 2) — fully built, but one-shot

- `app/Services/EmailCampaign/EmailCampaignService.php` — create / schedule / send-now, with an empty-audience guard.
- `app/Services/EmailCampaign/AudienceFilterResolver.php` — already does **RFM-style segmentation**: `min_orders`, `last_order_after/before`, `order_activity`, `account_type`, `tag_ids`, `newsletter`, `verified`, product-bought. This is reusable as journey *audience scope* and *exit conditions*.
- `app/Services/EmailCampaign/CustomerSegmentService.php` — saved segments with a cached `last_count`.
- `app/Jobs/EmailCampaign/SendEmailCampaignJob.php` — dispatch worker.
- Scheduled by `email-campaigns:dispatch-scheduled` (everyMinute) in `routes/console.php`.

This is **broadcast / one-shot**, not a multi-step journey. There is no concept of “step 2 of 3” or “wait 24h then send”.

3.2 Single-shot automation engine (Audience 2) — closest thing to triggered email

- `app/Services/Automation/TriggerDispatcher.php::dispatch(trigger, Model, tenant)` → finds enabled `Automation` rows → `ConditionValidator` → `ProcessAutomationJob` → `ActionExecutor`. ~25 confirmed dispatch call sites (`OrderService.php:82` ; `Order/Invoice/Quote` model boot hooks; `Quote/Invoice/Order/Payment/PrintJob` controllers; 3 scheduled commands).
- `app/Services/Automation/ActionExecutor.php` — does `[token]` placeholder parsing (`parse()` at line ~177) over customer/billing/shipping/job relations, sends via `EmailService::mailer` with `trigger_source='automation'` metadata; also supports SMS via `SmsService`. **Fires all actions synchronously inside one job — no delay handling.**
- `app/Jobs/ProcessAutomationJob.php` — queue worker.
- `app/Models/Automation.php` — `BelongsToTenant` + `$guarded = []`.

Schema is thin (`database/migrations/2025_05_26_060514_create_automations_table.php`): only `tenant_id`, `name`, `trigger` (string), `conditions` (json), `actions` (json), `action_details` (json), `enabled`, `sort_order`. The `2026_06_05` migration added `last_run_at` + `run_count`. **No delay / step / sequence / branch / enrolment columns.**

Trigger vocabulary is business-document only (`app/Enums/Automation/TriggerTypeEnum.php`): `status_changed`, `paid_in_full`, `approval_type_approved`, `approval_type_declined`, `task_completed`.

Action vocabulary (`app/Enums/Automation/ActionTypeEnum.php`): `send_email_text`, `request_approval`, `change_status` implemented; `assign_task` + `request_payment` are **DECLARED in the enum but NOT implemented** (the `ActionExecutor match()` has no arms for them → silent no-op). This is a \$0 “no half-built UI / silent-lie” violation a journey-action expansion should fix or hide.

3.3 Transactional email (Audience 1 + 2)

- `app/Services/EmailService.php` → `EmailProviderManager` (SendGrid → Postmark → SMTP fallback).
- 15 templates in `app/Enums/EmailTemplateEnum.php` (`ORDER`, `ORDER_SHIPPED`, `ORDER_STATUS_UPDATE`, `PAYMENT_FAILED`, `QUOTE_EXPIRY_REMINDER`, `ACCOUNT_CREATED`, ...) resolved store → tenant → none via `app/Services/Email/EmailTemplateResolverService.php`, seeded by `app/Jobs/Tenant/SeedTenantEmailTemplates.php`.

3.4 Platform → Tenant retention (Audience 1) — almost uninstrumented

- **The only proactive lifecycle nudge:** `app/Jobs/System/NotifyTrialExpiringJob.php` (scheduled dailyAt 08:00; notifies at the configured window + 1-day mark via `emails.system.trial-expiring`).
- Other tenant-lifecycle one-shots in `app/Jobs/System/`: `NotifyTenantApproved/Blocked/Deactivated/Reactivated`, `NotifyAdminPendingApprovals` — driven by `app/Services/Admin/AccountStatusService.php`.
- `app/Services/BillingService.php` has `renew()` but **no cron auto-invokes it**; `charge_failed` status is read by middleware but triggers **no email drip** (no dunning).

3.5 Building blocks already present for new lifecycle triggers

- **Abandoned cart:** `app/Models/Cart.php` + carts migration carry `tenant_id` , `customer_id` , `session_id` , `coupon_code` , `updated_at` — **sufficient to detect abandonment today.**
- **Dormancy / win-back:** `app/Models/Customer.php` has a `last_login_at` cast + `orders()` / `wishlists()` relations — sufficient for dormancy segmentation. `app/Models/Wishlist.php` exists; reorder route at `routes/frontstore.php:413` .
- **External-ESP escape hatch:** `app/Services/Integrations/Marketing/MarketingSyncService.php` → Mailchimp / Klaviyo with `trackEvent()` , pushed on `Customer::created` via `SyncCustomerToMarketingJob` (owners can run journeys in an external ESP today).
- **Scheduler seam:** `routes/console.php` already runs many everyMinute/daily crons (`email-campaigns:dispatch-scheduled` , `campaigns:dispatch-due` , `send:task-reminders` , `action-center:digest`). A new `journeys:tick` slots in here.
- **Fan-out:** `app/Jobs/Notification/SendAppNotification.php` (email/sms/push per subscription).

4. Gaps

1. **No journey/drip engine of any kind.** No model represents a multi-step sequence (trigger → wait → step → branch). The Automation engine is strictly event → one-action: no time-delay steps, no multi-step sequences, no branching, no per-recipient enrolment/state tracking.
2. **No lifecycle/behavioral triggers exist.** `TriggerTypeEnum` has no `cart_abandoned` , `order_delivered` , `customer_dormant` , `first_order_placed` , `milestone_reached` , `signup_completed` , `feature_not_adopted` . These events are never dispatched — the order conversation ends at `ORDER_SHIPPED` .
3. **No abandoned-cart recovery.** No detector job, no cart → email trigger, no recovery template, no journey. Cart rows accumulate and are never acted on. **Highest-ROI gap** (abandoned-cart ≈ 76% of automation revenue).
4. **No post-purchase / post-delivery journey.** No `order_delivered` event, no review-request email, no NPS/CSAT survey (confirmed: no model/job/template), no reorder reminder for consumable print products.
5. **No win-back / re-engagement / dormancy drip, and no frequency-capping or suppression layer.** Campaigns + automations + any future journey can each email the same customer with no global cap, no per-recipient send ledger, no cross-system unsubscribe/suppression check → deliverability + complaint risk.
6. **Platform → Tenant (Audience 1) is almost entirely uninstrumented.** Only `NotifyTrialExpiringJob` . No welcome/activation series, no go-live/setup-completion nudges, no feature-adoption prompts, **no dunning/failed-payment retry sequence** (`BillingService.renew()` exists but no cron invokes it; `charge_failed` triggers no email), no post-trial win-back. The tenant engine cannot serve Audience 1 — landlord-side plumbing must be built separately.

7. **Per-recipient enrolment state is absent.** No way to know a customer is on step 2 of 3, no de-dup (re-triggering re-enrols), no exit conditions (stop win-back if they order mid-sequence), no per-journey analytics (sent/opened/converted).
 8. **QUEUE=sync in dev/prod** (project memory). Any delayed-step engine **requires Redis with delayed dispatch** — the sync driver makes delays impossible and is a reliability risk. **This must be fixed first.**
 9. **assign_task + request_payment silently do nothing** — advertised in the enum/UI but the `ActionExecutor match()` default returns null. \$0 silent-lie violation; fix or hide while expanding journey actions.
-

5. Best practices & benchmarks

- **Behavioral triggers beat calendar drips ~3–5× on CTR** because they fire at the moment of intent/inaction. Treat email as a behavioral channel keyed to product events, not a broadcast calendar. — mailsoftly.com/blog/email-marketing-for-saas; tabular.email/blog/saas-email-sequences
 - **Lifecycle is a discipline:** Onboarding → Activation → Retention → Expansion/Win-back; each stage gets its own triggered sequence adapting to feature usage/milestones, with distinct messaging for engaged vs inactive users (especially trial conversion). — growth.cx/blog/saas-email-marketing-practices; mailsoftly.com
 - **Abandoned cart cadence:** first email within **30–60 min** of abandonment, then a **2–3 email series over ~3 days** (avg ~17h gap to email 2), each with a different angle (reminder → benefit → incentive). A 3-email series lifts conversion ~34%; cart emails drove ~76% of automation revenue; global cart abandonment hit ~75% in 2025. — omnisend.com/blog/abandoned-cart-email; targetbay.com/abandoned-cart-email; rejoiner.com
 - **Frequency capping is mandatory:** enforce a global per-recipient cap (opt-in lists ~4–8/month; reducing promo cadence can cut complaint rate 60%+) across **all** senders, plus sunset/suppression removing or quieting contacts after **60–90 days** of no engagement; clean lists quarterly. Capping + throttling + send-time optimization protect deliverability. — mailreach.co/blog/email-frequency-best-practices; validity.com; machine-marketing.com
 - **A real lifecycle platform combines** full-funnel automation, visual journey mapping, real-time behavioral triggers, and **lifecycle-stage state management** — i.e. journeys need per-contact enrolment state, not fire-and-forget. — saasemailplatforms.com/category/lifecycle-email; htmlemailbuilders.com
 - **SaaS trial-conversion sequences** should reinforce value, add social proof, and escalate urgency as the trial ends, branching on engagement — directly applicable to Print-Flow-360's 14-day trial (Audience 1). — mailsoftly.com; growth.cx
-

6. Recommended architecture for THIS codebase

Guiding principle: extend the existing automation substrate, don't reinvent it. Reuse `ActionExecutor.php` for step execution and the `[token] parse()` logic *verbatim*. Follow `CLAUDE.md`: `HasUuid + BelongsToTenant`, `FormRequest → Service → Resource`, business logic in `app/Services/Journey/`, composables in front-end, Pinia for state, `CacheServiceInterface` + `StoreHelper` (never the `Cache` facade), tests for every Service method.

6.1 New tenant-scoped models (Audience 2)

All three: `HasUuid + BelongsToTenant`. PostgreSQL schema (no MySQL-specific SQL; `enum()` compiles to a CHECK constraint).

Journey — the sequence definition.

Column	Type	Notes
<code>id</code>	bigint PK	
<code>uuid</code>	uuid, unique	routes/URLs use uuid (<code>CLAUDE.md</code>)
<code>tenant_id</code>	fk	<code>BelongsToTenant</code>
<code>name</code>	string	
<code>trigger</code>	string	new <code>TriggerTypeEnum</code> value
<code>conditions</code>	jsonb	reuse <code>AudienceFilterResolver</code> shape
<code>exit_conditions</code>	jsonb	e.g. stop win-back on <code>order_placed</code>
<code>audience_scope</code>	jsonb	optional segment scope
<code>enabled</code>	boolean, default false	seeded disabled (opt-in)
<code>last_run_at</code> , <code>run_count</code>	timestamp / int	mirror automations
timestamps / softDeletes		

JourneyStep — ordered steps.

Column	Type	Notes
<code>id</code> , <code>uuid</code> , <code>journey_id</code> (fk)		
<code>sort_order</code>	int	
<code>delay_minutes</code>	int	wait <i>before</i> this step (0 = immediate)
<code>action_type</code>	string	reuse/extend <code>ActionTypeEnum</code>
<code>action_details</code>	jsonb	reuse <code>ActionExecutor</code> token/email/SMS path

JourneyEnrolment — per-recipient state (the piece that's totally missing today).

Column	Type	Notes
<code>id</code> , <code>uuid</code> , <code>journey_id</code> (fk)		
<code>enrollable_type</code> / <code>enrollable_id</code>	morph	Customer or Cart
<code>current_step</code>	int	
<code>status</code>	string (CHECK)	<code>active</code> / <code>completed</code> / <code>exited</code>
<code>next_run_at</code>	timestamp, indexed	the tick query keys on this
<code>last_step_at</code>	timestamp	
<code>meta</code>	jsonb	dedup key, trigger snapshot

Index `(journey_id, enrollable_type, enrollable_id)` for **de-dup** (a re-trigger must not double-enrol an already-`active` recipient) and `(status, next_run_at)` for the tick query.

6.2 New triggers + dispatch sites

Add to `TriggerTypeEnum`: `cart_abandoned`, `order_delivered`, `first_order_placed`, `customer_dormant`. Wire `JourneyEnrolment` creation on dispatch (de-dup; honor `exit_conditions`). Keep `ProcessAutomationJob` untouched; add **`ProcessJourneyStepJob`** that executes the current step via `ActionExecutor`, advances `current_step`, computes the next step's `next_run_at = now + delay_minutes`, and **self-reschedules via delayed dispatch** (requires Redis — fix `QUEUE=sync` first).

6.3 Time-based detector cron

Add `journeys:tick` to `routes/console.php` (everyMinute, mirroring `email-campaigns:dispatch-scheduled`):

- (a) find carts with no associated order and `updated_at` older than the abandonment threshold → dispatch `cart_abandoned`;
- (b) find customers whose last order / `last_login_at` crosses a dormancy threshold → dispatch `customer_dormant`;
- © advance any `JourneyEnrolment` whose `next_run_at <= now` (belt-and-suspenders alongside delayed dispatch).

Use `CacheServiceInterface` + `StoreHelper`, never the `Cache` facade.

6.4 Service layer (`app/Services/Journey/`)

- `JourneyService` — CRUD on journeys/steps (FormRequest → Service → Resource); **audit-log enable/disable/edit** per `CLAUDE.md` significant-action rule.
- `JourneyEnrolmentService` — enrol (with de-dup), advance, exit, complete.
- `JourneyStepExecutor` — thin wrapper delegating to `ActionExecutor` so the token/email/SMS path is shared, not forked.

- Every method that contains business logic needs a test ([CLAUDE.md](#)). Mirror `tests/Feature/Storefront/AddressAttentionToTest.php` style: assert enrolment, delay scheduling, exit-on-purchase.

6.5 Frequency-cap / suppression layer (shared by all three senders)

- `EmailSuppression` model — per customer/email, `reason`, `scope`.
- `EmailSendLedger` model — `recipient`, `journey_id` / `campaign_id`, `sent_at`.
- `CommunicationGuard` service — consulted before **any non-transactional** send; enforces a per-recipient monthly cap and 60–90-day sunset. Route `SendEmailCampaignJob` and `ActionExecutor` through it. **Transactional email (order/payment) is exempt.**

6.6 Admin builder UI (`nuxt/` , Audience 2)

Mirror the existing automation form patterns (UI-consistency rule): trigger picker, ordered steps with **human delay labels** (“Wait 1 day”), live preview, per-journey analytics (enrolled / completed / converted from `JourneyEnrolment`). Plain-language empty/loading/error states (\$0). Wrap all API calls in composables (never `$fetch` in `.vue`); Pinia for state. While in this code, **fix the silent `assign_task` / `request_payment` actions** (implement or hide).

6.7 Audience-1 landlord plumbing (separate)

Build a thin landlord journey set as **scheduled System jobs** alongside `NotifyTrialExpiringJob` :

- new-tenant welcome/activation series,
- go-live/setup-completion nudges keyed to onboarding milestones,
- **dunning sequence** on `Subscription.status = charge_failed` (`BillingService.renew()` retry + escalation emails) — currently a hard gap,
- post-trial win-back.

Reuse `emails.system.*` views + `AccountStatusService` . **Do not** try to route these through the tenant `Journey` engine.

6.8 External-ESP option

Expose journey trigger events through `MarketingSyncService.trackEvent()` (`cart_abandoned` , `order_delivered` , `dormant`) so owners who prefer Klaviyo/Mailchimp can fire their own flows — choice without forcing the native engine.

7. Phased roadmap

Prerequisite (blocker)

- **Fix `QUEUE=sync` → Redis with delayed dispatch.** No delayed-step journey works on the sync driver. (See project memory: production readiness.) **~0.5–1 day** config + verification.

P0 — Native journey engine + flagship abandoned-cart (Audience 2)

- **P0.1** Journey / JourneyStep / JourneyEnrolment models + Postgres migrations. **~2–3 days**.
- **P0.2** New lifecycle triggers (`cart_abandoned` , `order_delivered` , `first_order_placed` , `customer_dormant`) + dispatch wiring with de-dup + exit conditions; `ProcessJourneyStepJob` . **~2–3 days**.
- **P0.3** `journeys:tick` cron (abandoned-cart + dormancy detectors). **~1–2 days**.
- **P0.4** Flagship **abandoned-cart recovery journey**: 3 steps (≈1h reminder → ≈24h benefit → ≈72h incentive), exit on `order_placed` . Seed **disabled** as a default in `StoreOnboardingService.php` . Add recovery template to `EmailTemplateEnum` + `SeedTenantEmailTemplates.php` . Tests: enrolment, delay scheduling, exit-on-purchase. **~2 days**.

P1 — Safety + post-purchase + landlord lifecycle

- **P1.1** `CommunicationGuard` + `EmailSuppression` + `EmailSendLedger` ; route campaigns + automations + journeys through it (transactional exempt). **~3 days**.
- **P1.2** Post-purchase journey: `order_delivered` → review-request → NPS/CSAT survey → reorder reminder. New `OrderFeedback` / `Survey` model + template (also seeds the absent feedback loop from `readme/qa_section_feedback.md`). **~3–4 days**.
- **P1.3** Audience-1 landlord journeys: welcome/activation, go-live nudges, **dunning on** `charge_failed` , post-trial win-back. **~3–4 days**.

P2 — Builder UI + ecosystem polish

- **P2.1** Admin journey builder UI in `nuxt/` with per-journey analytics; fix silent `assign_task` / `request_payment` . **~4–5 days**.
- **P2.2** Expose `cart_abandoned` / `order_delivered` / `dormant` via `MarketingSyncService.trackEvent()` ; audit logging on journey enable/disable/edit. **~1–2 days**.

8. Success metrics to track

Audience 2 (store → end-customer)

- Abandoned-cart **recovery rate** (% of `cart_abandoned` enrolments that hit the `order_placed` exit) — target the ~34% conversion lift benchmark.
- Recovered **revenue** attributed to journeys (ledger-joined to orders).
- Per-journey funnel: enrolled → step-completed → converted → exited.
- Review-request response rate; NPS/CSAT volume and score.
- Reorder-reminder conversion for consumable products.

Deliverability / safety

- Per-recipient **send frequency** distribution vs the cap (alert on breaches).
- **Complaint rate** and unsubscribe rate before/after capping (target 60%+ complaint reduction).

- Suppression/sunset list growth; % of sends blocked by `CommunicationGuard` .

Audience 1 (platform → tenant)

- **Trial → paid conversion** uplift from the activation/expiry series.
 - **Dunning recovery rate** (% of `charge_failed` subscriptions recovered).
 - Go-live completion rate among tenants who received setup nudges.
-

9. Key file references

Area	Path
Single-shot trigger dispatch	<code>app/Services/Automation/TriggerDispatcher.php</code>
Action execution + [token] parser (REUSE)	<code>app/Services/Automation/ActionExecutor.php</code>
Triggers enum (extend)	<code>app/Enums/Automation/TriggerTypeEnum.php</code>
Actions enum (silent gaps)	<code>app/Enums/Automation/ActionTypeEnum.php</code>
Automation worker	<code>app/Jobs/ProcessAutomationJob.php</code>
Automation model + thin schema	<code>app/Models/Automation.php</code> , <code>database/migrations/2025_05_26_060514_create_automations_table.php</code>
Broadcast campaigns	<code>app/Services/EmailCampaign/EmailCampaignService.php</code> , <code>.../SendEmailCampaignJob.php</code>
RFM segmentation (REUSE for scope/exit)	<code>app/Services/EmailCampaign/AudienceFilterResolver.php</code> , <code>.../CustomerSegmentService.php</code>
Transactional email	<code>app/Services/EmailService.php</code> , <code>app/Services/Email/EmailTemplateResolverService.php</code> , <code>app/Enums/EmailTemplateEnum.php</code> , <code>app/Jobs/Tenant/SeedTenantEmailTemplates.php</code>
Audience-1 retention	<code>app/Jobs/System/NotifyTrialExpiringJob.php</code> , <code>app/Services/Admin/AccountStatusService.php</code> , <code>app/Services/BillingService.php</code>
Trigger building blocks	<code>app/Models/Cart.php</code> , <code>app/Models/Customer.php</code> , <code>routes/frontstore.php:413</code>
Onboarding seed point	<code>app/Services/Onboarding/StoreOnboardingService.php</code>
External ESP	<code>app/Services/Integrations/Marketing/MarketingSyncService.php</code>
Fan-out	<code>app/Jobs/Notification/SendAppNotification.php</code>
Scheduler seam	<code>routes/console.php</code>
Feedback-loop gap	<code>readme/qa_section_feedback.md</code>
Queues reference	<code>readme/NOTIFICATIONS_AND_QUEUES.md</code>

NPS / CSAT / Customer Health Scoring & At-Risk Alerting

Source: [readme/research/retention-lifecycle/RETENTION_NPS_CSAT_HEALTH_SCORING_2026-06-16.md](#)

IN SHORT — KEY TAKEAWAYS

This document is about knowing when a customer is about to stop buying from you — before they actually leave. It explains how to measure whether customers are happy, spot warning signs early, and automatically nudge the store owner to take action. Right now the platform has none of this in place, so it is effectively flying blind.

The main parts explained simply:

- **NPS (Net Promoter Score)** — A single question: “How likely are you to recommend us?” scored 0–10. Promoters (9–10) are loyal fans; detractors (0–6) are unhappy and likely to leave. The gap between the two is your score. Send this a week or two after a customer receives their order.
- **CSAT (Customer Satisfaction Score)** — A quick rating right after a specific moment: finishing setup, getting help, or placing an order. Tells you how that one experience felt. A good target is 75–85% positive; top businesses aim for 90%+.
- **CES (Customer Effort Score)** — Measures how easy or hard it was to do something (like placing an order). Lower effort = happier customers = more repeat business.
- **Health Score** — A single number (0–100) that combines several signals about a customer: how often they order, how recently they logged in, whether a payment has failed, and how they rated past experiences. Green (75–100) = healthy; Yellow (40–74) = watch this person; Red (0–39) = act now. The key insight is that 70–80% of customers who eventually leave show warning signs 30–60 days before they go.
- **Two audiences** — The platform needs to track two groups separately: (1) store owners who might cancel their subscription to the platform, and (2) buyers who might stop ordering from a store. Both need health scores; both need early warnings.
- **At-Risk Alerts** — Instead of making a store owner read charts, a score drop automatically creates a plain-language nudge in the Action Center (“Maria rated her last order 4/10 — follow up before she leaves”). No new screens; the store owner just acts on the prompt they already see.
- **What already exists** — The codebase already has the alert system (Action Center), email sending, and a basic “hasn’t ordered in 90 days” check. The missing pieces are: a way to send surveys, a proper multi-signal health score, and the logic to turn a low score into an alert.
- **Survey timing matters** — Send NPS 1–2 weeks after a delivery. Send CSAT right after setup or support. Embedding the first survey question directly in the email (not just a link) gets far more replies.

What to do with this: Build the survey system and health score first (P0 priority), then wire low scores to automatic Action Center alerts so store owners get a plain-English prompt to follow up — without ever needing to look at a dashboard. Do not turn on nightly background jobs until the queue system is switched to Redis (currently it runs in a mode that is not reliable enough for scheduled tasks).

Date: 2026-06-16 **Status:** research-only (nothing built yet) **Scope:** Both audiences — (1) the platform retaining its *tenant store owners* (B2B SaaS churn), and (2) each *store owner* retaining their *end-*

customers (buyer churn).

TL;DR

Print-Flow-360 currently has **no way to measure customer satisfaction or predict churn** for either audience. There is no survey/NPS/CSAT/CES spine, no composed health score, and no at-risk-tenant feed for the platform's own retention. What *does* exist is a strong foundation to build on: the **Action Center** rule engine (`app/Services/ActionCenter/`) is the exact pattern for at-risk alerting and already ships a crude end-customer churn proxy (`AtRiskCustomersRule` — 90-day recency). Survey *delivery* rails already exist (EmailService, the Automation trigger/action engine, in-app `SendAppNotification` , the daily scheduler). The recommended path: build one shared **survey spine** (`Survey` + `SurveyResponse` models + `SurveyService`), wire delivery through the *existing* email/automation/in-app rails (no new engine), compose two **HealthScoreService** classes (end-customers first, tenants second), persist nightly scores, and turn score-band changes into **Action Center alerts** so a non-technical store owner gets a “follow up with this customer” nudge with zero new screens.

1. Why this matters for a non-technical-store-owner print SaaS

The primary user is a non-technical store owner who navigates by intuition and will not configure analytics dashboards. For them, retention has to arrive as **a plain-language nudge in a place they already look**, not a metric they have to go hunt for:

- *“Acme Print Co. hasn’t ordered in 112 days and has a failed payment — reach out.”*
- *“Maria rated her last order 4/10 — follow up before she leaves.”*

That is exactly the **Action Center** experience this codebase already ships (a computed “what needs action now” feed). Health scoring and detractor surveys only earn their keep when a score change *triggers* one of those nudges — assign a task, raise an alert, fire a win-back email — so the owner never has to read a number.

For the **platform itself**, the same logic applies one level up: the landlord (SaaS operator) needs to know which *tenants* are going quiet, failing payments, or trending toward cancellation **30–60 days before they churn**, because 70–80% of churning customers show measurable warning signs that far ahead. Today the platform is effectively blind to this — the only proactive tenant-retention touch is

`NotifyTrialExpiringJob` .

2. The two audiences — and which applies where

	Audience 1 — Platform → Tenant	Audience 2 — Store owner → End-customer
Who churns	A store owner cancels their subscription	A buyer stops ordering from a store
Who acts	Platform CS / landlord	Store owner / staff
Signal sources	<code>Tenant.status</code> , <code>Subscription</code> (trial/active/expired/charge_failed), staff activity, products published, orders processed, stores live	<code>Customer.last_login_at</code> / <code>login_count</code> , orders (recency/frequency/value), failed <code>Payment</code> , reviews/messages, survey sentiment
Surface	NEW landlord health dashboard (<code>routes/admin-api.php</code> + Nuxt page) + at-risk-tenant alert	Existing tenant Action Center feed (<code>AtRiskCustomersRule</code> evolved)
Instrumentation today	Least-instrumented — no health band, no at-risk feed	Partly seeded — single-signal recency rule exists
Survey type fit	Relational NPS 7–10d post-onboarding; CSAT post-onboarding; CSAT post-support (once support spine exists)	Transactional NPS after delivery; CSAT post-support; relational NPS at intervals

Both audiences share **one survey spine** and the **same `HealthScoreService` shape** (different signal sets and weights). Keep the tenant-facing health surface separate from the tenant Action Center, since Action Center is tenant-scoped and the at-risk-tenant feed is for the landlord.

3. What already exists in the codebase

3.1 The at-risk alerting pattern is ready — copy it

- `app/Services/ActionCenter/ActionCenterService.php` — runs rules registered in `rules()` (currently 8: OverdueInvoices, FailedPayments, JobsDue, ProofsAwaiting, ExpiringQuotes, LowStock, **AtRiskCustomers**, B2bApprovals). **Adding a rule = 1 class + 1 line.**
- `app/Services/ActionCenter/AlertRule.php` — the interface. Each rule returns items with `type / severity / title / subtitle / link / icon / entity_type / entity_id / date`.
- `readme/ACTION_CENTER.md` — canonical doc and the rule-authoring SOP.
- `nuxt/app/components/dashboard/DashboardActionCenterWidget.vue` — the dashboard surface; reuse for satisfaction/health widgets.

3.2 An end-customer churn proxy already ships (the seed to evolve)

- `app/Services/ActionCenter/Rules/AtRiskCustomersRule.php` — `info` severity; customers with orders ever but none in 90+ days, using `withMax` `last_order_at` + `withCount` orders. This is a **single-signal recency heuristic, not a composed health score** — but it is exactly the rule to evolve into a band-driven alert.

3.3 End-customer signals already captured

- `app/Models/Customer.php` + `database/migrations/2026_06_04_000002_add_last_login_to_customers_table.php` — `last_login_at` + `login_count` . (Per `readme/CUSTOMER_ACTIVITY_AND_SOURCE.md` : must increment via `login_count` , `[last_login_at => now()]` or it silently drops.)
- Orders relationship → count / recency / value. `Payment.status = failed` feeds `app/Services/ActionCenter/Rules/FailedPaymentsRule.php` .

3.4 Tenant (platform-churn) signals already captured

- `app/Models/Tenant.php` — `status` (ACTIVE/TRIAL/INACTIVE), `trial_approval_date` , `subscriptions()` .
- `app/Models/Subscription.php` — `status` (trial/active/expired/cancelled/charge_failed), `trial_ends_at` , `ends_at` , `renewal_at` .
- `app/Services/Admin/AccountStatusService.php` — drives tenant lifecycle.
- `app/Jobs/System/NotifyTrialExpiringJob.php` — the **only** proactive tenant-retention nudge today.

3.5 Survey delivery rails already exist (no new engine needed)

- **Email:** `app/Services/EmailService.php` → `EmailProviderManager`; templates via `app/Services/Email/EmailTemplateResolverService.php` + `app/Enums/EmailTemplateEnum.php` , seeded by `SeedTenantEmailTemplates` .
- **Triggered single-shot:** Automation engine `app/Services/Automation/TriggerDispatcher.php` (~25 call sites incl. `OrderService:82` , `status_changed` / `paid_in_full`) → `ProcessAutomationJob` → `app/Services/Automation/ActionExecutor.php` (`send_email_text`).
- **In-app multi-channel:** `app/Jobs/Notification/SendAppNotification.php` (email/sms/push).
- **Scheduler:** `routes/console.php` (`action-center:digest` dailyAt 07:30 / weeklyOn Mon) — where a time-based survey/health cron slots in.

3.6 Supporting pieces

- `app/Models/Activity.php` (`user_id` , `subject` , `description`) — the mechanism to log health-band changes and survey sends (CLAUDE.md audit rule).
- `app/Services/Onboarding/StoreOnboardingService.php` — `seedAll` is the anchor for a post-onboarding CSAT timer.

3.7 Feedback signals that currently rot (should feed a score, but go to a void)

- **ProductReview** — `status = pending` , no owner notification.
 - **OrderMessage** thread — customer messages reach a void, no admin reply panel.
 - Storefront contact form → `StoreFormSubmission` (write-only; no ticket, no ref number) — confirming **no support spine** to source CSAT from.
-

4. Gaps

1. **No survey model/migration/job/template** for NPS (relational or transactional), CSAT, or CES — for either audience. Satisfaction cannot be measured at all today.
 2. **No composed health score.** `AtRiskCustomersRule` uses one 90-day recency signal — no feature-adoption, payment-health, support-ticket, or sentiment inputs combined.
 3. **No tenant health score at all** (the platform's own B2B SaaS churn — the least-instrumented audience). No band, no at-risk-tenant alert, no dunning beyond the `EnsureTenantHasActiveSubscription` block + grace banner.
 4. **No survey-timing trigger.** `TriggerTypeEnum` has only `status_changed` / `paid_in_full` / `approval/task_completed` — **no** `order_delivered` → transactional NPS can't fire on fulfilment without a new trigger + dispatch site. No onboarding-complete or post-support trigger.
 5. **No support ticket spine** → post-support CSAT has nothing to attach to (`readme/qa_section_support.md` : no Ticket model/controller/SLA).
 6. **No response storage, aggregation, trend, or satisfaction/health dashboard** on either side.
 7. **No Action Center rule consuming health bands or detractor responses**; no CS/landlord-facing at-risk-tenant surface (`routes/admin-api.php` has no health/metric route — only dashboard-welcome settings).
 8. **Reliability risk:** `QUEUE=sync` per memory (must be Redis in prod) before scheduling nightly score jobs; failed-payment/dunning is passive and not folded into any score, despite card failure causing 20–40% of SaaS churn.
 9. **Thin adoption signal:** `login_count` / `last_login_at` is the only persisted end-customer adoption signal; there is no per-tenant feature-adoption signal (would be computed from existing `products` / `orders` / published pages).
-

5. Best practices & benchmarks

Use all three metrics, each for a different moment (Delighted; Balto; Armatis 2025):

- **CSAT** immediately after a discrete interaction (onboarding, support resolution).
- **CES** immediately after task completion.
- **NPS** for loyalty at broader intervals. Don't pick one.

Survey timing that maximizes response (Zonka Feedback tNPS guide 2025):

- Transactional NPS 0–24h for quick events; **1–2 weeks after a delivery**.
- SaaS relational NPS **7–10 days post-onboarding**.
- Relational NPS quarterly for B2B, semi-annual for B2C.

Channel response-rate benchmarks (Zonka Feedback 2025):

Channel	Response rate
SMS	40–50%
In-app mobile	27–36%
In-app web survey	20–27%
Email, embedded first question	15–25%
Email, link only	6–15%
Transactional NPS gets 8–12 pts higher response than relational (25–40% vs 15–25%). Embed the first question in the email ; prefer in-app for the captive moment.	

CSAT target: 75–85% is good; SaaS top performers target 90%+ (Balto/Delighted 2025).

Health score = weighted multi-signal, not a single metric. Signal-Stack model: **Activity 40% / Engagement 30% / Milestones 20% / Recency 10%** — usage decline is the single strongest churn predictor, ahead of NPS, tickets, and sentiment. Low NPS + declining usage *together* is far stronger than either alone (EverAfter 2025; Accoil; [customerscore.io](#)).

Payment/dunning health is a first-class input — card failures cause **20–40% of SaaS churn**; fold failed-payment / charge_failed / expired-card state into the score and act on it ([customerscore.io](#); EverAfter).

Health bands (3-tier): Green 75–100 / Yellow 40–74 / Red 0–39 (variant: Green 70–100 / Yellow 40–70 / Red <40). Pick one and **backtest against historical churn** ([hellopmpm.co](#); HubSpot; Custify 2025).

A score only earns its place when a change TRIGGERS an action — for each threshold define action / owner / channel / timing and automate it (triggered email, alert, CSM assignment) (DigitalApplied 2026; Totango).

Leading-indicator timing: ~70–80% of churning customers show measurable warning signs **30+ days before** cancelling; activity decline precedes cancellation by **30–60 days**; predictive models give 60–90 days warning. **Score nightly and alert on the down-trend, not just the current value** (DigitalApplied 2026; EverAfter).

Maintain the model: re-weight monthly, review accuracy quarterly, reassess metrics semi-annually, full re-baseline annually; backtest weights against past churn (mean.ceo 2026; EverAfter).

6. Recommended architecture for THIS codebase

All new code follows [CLAUDE.md](#): UUID routes (`HasUuid`), `BelongsToTenant` for tenant-scoped models, `Controller` → `FormRequest(BaseRequest)` → `Service` → `Resource` → `successResponse()/errorResponse()` , business logic in `app/Services/{Module}/` , composables wrapping every API call in Nuxt, Pinia for state, Postgres-compatible migrations (`enum()`) → CHECK constraint per §6), and a Feature test proving each submitted field round-trips (silent-lie rule).

6.1 Shared survey spine — `app/Services/Feedback/`

New tenant-scoped models (`BelongsToTenant` + `HasUuid`):

- **Survey** — `id`, `uuid`, `tenant_id`, `type` (`nps|csat|ces` → Postgres CHECK), `audience` (`end_customer|tenant`), `trigger` (`order_delivered|post_onboarding|post_support|relational`), `schedule` (nullable cadence), `is_active`, timestamps.
- **SurveyResponse** — `id`, `uuid`, `tenant_id`, `survey_id`, `customer_id` or `tenant_subject_id` (one set), `score` (0–10 NPS / 1–5 CSAT-CES), `comment` (text), `source_channel` (`email|in_app|sms`), `context_snapshot` (JSON — snapshot of trigger context so a renamed product/trigger doesn't lose meaning; snapshot rule), `responded_at`, timestamps.

Service: `app/Services/Feedback/SurveyService.php` — compose surveys, store responses, compute:

- NPS = `%promoters (9–10) - %detractors (0–6)`.
- CSAT = `% scoring 4–5 (or top-2-box)`.
- CES = mean effort score.
- Classification helper: `isDetractor(response)` (NPS ≤6 or CSAT ≤2) → drives §6.5 action wiring.

API: `SurveyController` + `SurveyResponseController` (UUID routes),

`StoreSurveyRequest` / `StoreSurveyResponseRequest` extending `BaseRequest`,

`SurveyResource` / `SurveyResponseResource`. **Feature test:** submit a response, assert it persists (model

`Survey.php` -style test mirroring `tests/Feature/Storefront/AddressAttentionToTest.php`).

6.2 Survey delivery via existing rails (no new engine)

- **Transactional end-customer NPS:** add `order_delivered` to `app/Enums/Automation/TriggerTypeEnum.php` + a `TriggerDispatcher::dispatch('order_delivered', ...)` call site where a job/order hits its terminal/delivered status. Then send via a survey action — extend `ActionExecutor` with a `send_survey` action or reuse `send_email_text` carrying a survey link.
- **In-app + email channel:** render via `EmailTemplateResolverService` (new `EmailTemplateEnum` slug) and a storefront one-question NPS widget in `frontstore/` (embed the first question per benchmarks). Multi-channel send through `SendAppNotification`.
- **Post-onboarding CSAT (store owner):** new `app/Jobs/System/SendTenantOnboardingCsatJob.php` (sibling of `NotifyTrialExpiringJob`), fired **7–10 days after** `StoreOnboardingService::seedAll`, delivered via `EmailService`.

6.3 `CustomerHealthService` — Audience 2 (end-customers) — P0

`app/Services/Health/CustomerHealthService.php` composing weighted signals already available:

Signal group (Signal-Stack weight)	Source
Activity 40%	<code>orders_count</code> , <code>last_order_at</code> , <code>last_login_at</code> , <code>login_count</code>
Engagement 30%	reviews / <code>OrderMessage</code> activity, survey responses
Milestones 20%	order totals (monetary), repeat-purchase
Recency 10%	days since last order/login
Negative weight for failed <code>Payment</code> count (dunning input).	

Persist nightly to a new Postgres table `customer_health_scores` (`uuid` , `tenant_id` , `customer_id` , `score` , `band` , `signals` JSON, `computed_at`). New cron in `routes/console.php` next to `action-center:digest` . Bands: **Green 75–100 / Yellow 40–74 / Red 0–39**.

6.4 Evolve `AtRiskCustomersRule` into a health-driven alert — P0

Augment `app/Services/ActionCenter/Rules/AtRiskCustomersRule.php` to read the persisted band (Red → `warning` , Yellow → `info`) instead of the lone 90-day check, with a richer subtitle: “*Health: At risk · last order 112d · 1 failed payment.*” Pure Action Center pattern — one class, **names not ids**, `>limit(50)` , null-guard deleted relations (`relation?->field ? : 'Deleted X'`), `CurrencyHelper` for money. Gives the store owner a CS-style at-risk feed with **zero new frontend**.

6.5 Wire detractor responses into action — P1

When `SurveyService::isDetractor()` is true:

- (a) Create an Action Center alert (“*Detractor: Acme rated 4/10 — follow up*”) for the store owner.
- (b) Log to `Activity` (audit rule).
- © For tenant surveys, alert the platform CS/landlord.

6.6 `TenantHealthService` — Audience 1 (platform’s own churn) — P1

`app/Services/Admin/TenantHealthService.php` (landlord side). Signals from `Tenant.status` / `trial_approval_date` , `Subscription` (`charge_failed` / `expired` / `trial_ends_at`), staff last-activity, tenant productivity (products published, orders processed, stores live). Bands + nightly cron → new Postgres table `tenant_health_scores` . Surface in a **new landlord health dashboard**: add a route to `routes/admin-api.php` (none exists today) + a Nuxt page in `nuxt/` with an at-risk-tenant list (separate from tenant Action Center).

6.7 Feed dunning into both scores — P1

Card failure = 20–40% of churn. **Tenants**: detect `charge_failed` / `expired` and trigger a “payment failed — update your card” email sequence via the scheduler (`renew()` has no auto-cron today). **End-customers**: add failed- `Payment` count as a weighted negative in `CustomerHealthService` (`FailedPaymentsRule` already alerts).

6.8 Post-support CSAT — P2 (blocked)

Blocked on building a `Ticket` model (`readme/qa_section_support.md`). When a ticket resolves, fire a 1-question CSAT via `SurveyService` . Until then, attach CSAT to `OrderMessage` thread resolution and `ProductReview` submission — and **wire the missing owner-notification** so reviews/messages stop rotting.

6.9 Trend dashboards — P2

- **Tenant-side:** satisfaction widget (NPS trend, response rate, detractor list) reusing the `DashboardActionCenterWidget.vue` pattern.
- **Landlord-side:** tenant health distribution (Green/Yellow/Red counts) + at-risk list. Both must ship loading/empty/error states (`CLAUDE.md` §0) and **paginate** the at-risk lists (pagination-required rule). Every API call wrapped in a composable; state in Pinia.

7. Phased roadmap & effort sizing

Phase	Deliverable	Effort	Depends on
P0	Survey spine: <code>Survey</code> + <code>SurveyResponse</code> models/migrations, <code>SurveyService</code> , controllers/requests/resources, Feature test	M (3–5d)	—
P0	Survey delivery: <code>order_delivered</code> trigger + dispatch site, <code>send_survey</code> / <code>send_email_text</code> action, email template slug, in-app NPS widget, <code>SendTenantOnboardingCsatJob</code>	M (3–5d)	survey spine
P0	<code>CustomerHealthService</code> + <code>customer_health_scores</code> table + nightly cron	M (3–4d)	— (uses existing signals)
P0	Evolve <code>AtRiskCustomersRule</code> to read health band	S (1d)	health scores persisted
P1	Detractor → Action Center alert + Activity log	S (1–2d)	survey spine + Action Center
P1	<code>TenantHealthService</code> + <code>tenant_health_scores</code> + landlord health dashboard (admin-api route + Nuxt page)	L (5–8d)	—
P1	Dunning/payment-health into both scores + tenant card-failure email sequence	M (3–4d)	tenant health
P2	Post-support CSAT	M	Ticket model (separate epic)
P2	Satisfaction + health trend dashboards	M (3–5d)	scores + responses
Cross-cutting	<code>QUEUE=redis</code> before nightly jobs; log every band transition + survey send to <code>Activity</code> ; snapshot survey context; re-weight monthly + backtest	ongoing	—

Reliability gate: do **not** schedule the nightly score crons until `QUEUE=redis` is confirmed in prod (sync queue is a reliability risk per memory).

8. Success metrics to track

Measurement coverage (does the spine work):

- Survey response rate by channel vs benchmarks (in-app 20–27%, email-embedded 15–25%, SMS 40–50%).
- % of delivered orders that trigger a transactional NPS; % of onboardings that fire CSAT.

Satisfaction (the numbers themselves):

- End-customer & tenant **NPS** (%promoters – %detractors), trended monthly.
- **CSAT** (target 75–85%, stretch 90%+) post-onboarding and post-support.
- Detractor follow-up rate (% of $\leq 6/\leq 2$ responses that get an Action Center alert *and* an owner action logged in `Activity`).

Health-score efficacy (does it predict churn):

- Band distribution (Green/Yellow/Red counts) per audience, trended.
- **Backtest accuracy:** of customers/tenants who churned, what % were Red/Yellow 30–60 days prior (target the 70–80% leading-indicator benchmark).
- Lead time: median days between first Red band and churn (target 30–60+).

Retention outcome (the point of it all):

- End-customer repeat-order rate and tenant subscription renewal/retention rate, segmented by whether an at-risk alert fired and was actioned.
- Reduction in payment-failure-driven churn after the dunning sequence ships.

9. Key file references

- `readme/ACTION_CENTER.md`
- `app/Services/ActionCenter/ActionCenterService.php` , `AlertRule.php` ,
`Rules/AtRiskCustomersRule.php` , `Rules/FailedPaymentsRule.php`
- `app/Models/Customer.php` · `readme/CUSTOMER_ACTIVITY_AND_SOURCE.md` ·
`database/migrations/2026_06_04_000002_add_last_login_to_customers_table.php`
- `app/Models/Tenant.php` · `app/Models/Subscription.php` ·
`app/Services/Admin/AccountStatusService.php` · `app/Jobs/System/NotifyTrialExpiringJob.php`
- `app/Services/EmailService.php` · `app/Services/Email/EmailTemplateResolverService.php` ·
`app/Enums/EmailTemplateEnum.php`

- [app/Services/Automation/TriggerDispatcher.php](#) · [app/Services/Automation/ActionExecutor.php](#) · [app/Enums/Automation/TriggerTypeEnum.php](#)
- [app/Jobs/Notification/SendAppNotification.php](#) · [app/Models/Activity.php](#)
- [routes/console.php](#) · [routes/admin-api.php](#) · [app/Services/Onboarding/StoreOnboardingService.php](#)
- [readme/qa_section_feedback.md](#) · [readme/qa_section_support.md](#)
- [nuxt/app/components/dashboard/DashboardActionCenterWidget.vue](#)